Congestion-Aware Adaptive Traffic Routing Using Dynamic Graph Optimisation and Neural Network Prediction

Harvey Humphrey

Independent research dissertation prepared in support of postgraduate applications

Completed during studies toward the degree of

BSc Computer Science with Artificial Intelligence

Oxford Brookes University

2026

Abstract

Controlling congestion in complex road networks poses a difficult challenge due to high numbers of vehicles occurring on roads with limited alternative routes. In many cases, highly used roads serve as preferred route choices for vehicles as they offer the most straightforward way to connect origin and destination points. When vehicle volumes increase, congestion occurs on bottlenecked locations such as motorways which increases travel-time for each vehicle.

One of the possible ways to reduce congestion is using adaptive re-routing which diverts vehicles towards alternative routes so that not all vehicles are utilising the same roads which usually would cause congestion bottlenecks. Shortest-path algorithms such as Dijkstra's algorithm and A* are often used within graph-based routing systems as they are able to compute routes in the graphs network.

These algorithms cannot solely be applied to realistic models of transportation as they do not consider dynamic traffic conditions, such as congestion, road closures and accidents. The limitations become even more critical when applied to populated urban areas with many intersections and limited possible routes.

To address these issues, a system was created using real-world network data sourced from OpenStreetMap through the OSMnx library. The road network was then modelled as a weighted directed graph where intersections were considered as nodes and roads were shown as weighted edges. These nodes and edges held travel-time and congestion measurements. Routes for vehicles were then calculated using both Dijkstra's algorithm and the A* pathfinding algorithm to examine their performance under varying traffic loads.

Congestion was simulated using an edge-utilization measure, which meant that frequently utilised roads gained an increased travel-time costs during simulation. The road network adapted according to the traffic situation and allow for re-routing. A custom feed-forward neural network implementation based on basic principles which was trained to learn congestion multipliers using features such as edge utilisation, road length and road classification information.

Evaluation of the traffic-routing system was performed under higher traffic volumes which allowed us to examine the effects of routing scalability and congestion distribution across the network. By comparing Dijkstra's algorithm and A*, the analysis showed that heuristic-guided routing reduced unnecessary graph exploration and improved travel-cost efficiency under more complex traffic conditions. While both algorithms converged toward similar connected roads, suggesting how road formats strongly influences congestion regardless of the routing algorithm implemented.

The neural network converged consistently during training and learned general congestion patterns from the generated data. Prediction accuracy was strongest under lower and moderate congestion conditions. Performance became less reliable under severe congestion,

partly due to imbalance within the training dataset. Despite these limitations, the model successfully learned congestion prediction and can influence adaptive route selection within dynamically weighted transportation systems.

Overall, the dissertation demonstrates how graph-based optimisation and neural-network-based congestion prediction can be integrated within adaptive traffic-routing systems while also showing multiple practical limitations associated with large-scale congestion modelling, constrained road topology, and real-world transportation complexity.

Contents

List of Figures

List of Tables

Chapter 1

INTRODUCTION

1.1 Background

Traffic congestion continues to be a significant challenge within dense urban transportation systems, particularly within densely populated urban regions such as London and Manchester where population density is significantly higher Chang et al. (2017). Congestion reduces transportation efficiency and increases travel time for commuters moving between destinations.

With an increase in travel times and congestion, this introduces financial costs due to greater fuel consumption, prolonged idle periods, and inefficient re-routing behaviour. These effects become increasingly significant as urban populations continue to grow and transportation infrastructure becomes more heavily utilised.

This dissertation investigates whether traffic routing using graph optimisation and neural-network-based congestion prediction can improve traffic distribution compared to classical routing algorithms within structurally constrained urban road networks. Classical shortest-path approaches such as Dijkstra's algorithm and A* are evaluated as foundational routing methods Foead et al. (2021). Shortest-path algorithms alone may not provide globally efficient traffic flow due to dynamic traffic constraints. These approaches are extended through adaptive congestion weighting and a custom feed-forward neural network capable of adapting to surrounding traffic conditions and constraints.

1.2 Project Aim

The primary objective of the project is to determine whether congestion-aware routing combined with learned traffic prediction can improve overall traffic distribution and reduce bottleneck severity across weighted road-network graphs.

1.3 Research Objectives

- Implement weighted graph-based traffic routing

- Compare Dijkstra and A* algorithms
- Develop congestion-aware edge weighting
- Simulate multiple vehicles across real-world road networks
- Create congestion heat-map visualisations
- Implement a custom feed-forward neural network
- Predict congestion multipliers using traffic features
- Evaluate routing and neural-network performance

1.4 Project Contributions

The primary contributions of this dissertation include:

- Development of a congestion-aware adaptive traffic-routing framework using graph optimisation
- Comparative evaluation of Dijkstra and A* routing algorithms under dynamically evolving congestion conditions
- Implementation of adaptive congestion weighting using edge-utilisation metrics
- Creation of congestion heat-map visualisations for traffic-flow analysis
- Design and implementation of a custom feed-forward neural network from first principles without external machine-learning frameworks
- Integration of learned congestion prediction into adaptive route optimisation
- Experimental evaluation of routing scalability, bottleneck formation and neural-network regression performance

Chapter 2

LITERATURE REVIEW

2.1 Graph Theory and Road Networks

Road networks can be represented mathematically as weighted graphs where intersections are represented as nodes and roads are represented as weighted edges.

OSMnx was responsible for extracting real-world road-network data directly from **OpenStreetMap**. Functions such as `graph_from_point()` and `graph_from_place()` were used to generate the Oxford road network, including road layouts, intersections, geographic coordinates, and road metadata. As a result the system to model realistic urban road structures without manually constructing graph data, which would have been computationally inefficient and outside the scope of the dissertation.

The extracted road network was converted into a `MultiDiGraph` structure compatible with NetworkX. OSMnx additionally provided latitude and longitude coordinates alongside edge attributes such as road length and highway classification. These values enabled realistic route plotting, congestion heat-maps, and Euclidean-distance heuristics used within the A* algorithm. A `MultiDiGraph` structure was selected because real transportation systems frequently contain multiple directed road segments, lane variations and one-way connections between the same intersections.

NetworkX was then used to perform graph-theoretic operations on the generated road network. Hagberg et al. (2008) This included shortest-path routing, graph traversal, weighted edge calculations, and congestion manipulation.

Dijkstra's algorithm was implemented using the NetworkX function:

```
nx.shortest_path()
```

while A* routing was implemented using:

```
nx.astar_path().
```

These algorithms handled shortest-path calculations, weighted routing behaviour, and vehicle traversal across the network graph.

NetworkX additionally enabled iteration through edges and nodes using structures such as:

```
for u, v, key in G.edges(keys=True):
```

This enabled the project to calculate edge usage frequencies, dynamically update congestion weights, store travel-time costs, and apply route changes. The graph representation of the Oxford road network is shown in Figure 2.1. Intersections are represented as graph nodes, while roads are represented as directed weighted edges. Edge weights dynamically change according to congestion conditions and travel-time multipliers generated during simulation.

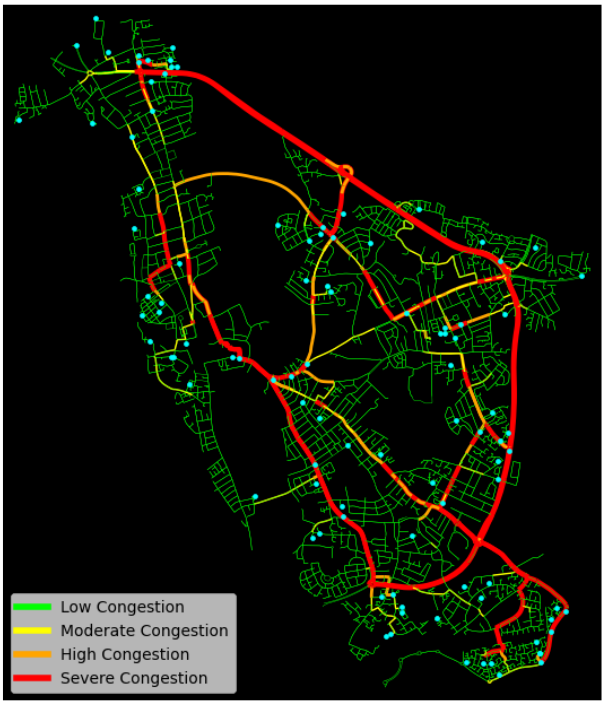

Figure 2.1: Weighted road-network congestion heat-map before dynamic re-routing using congestion weights

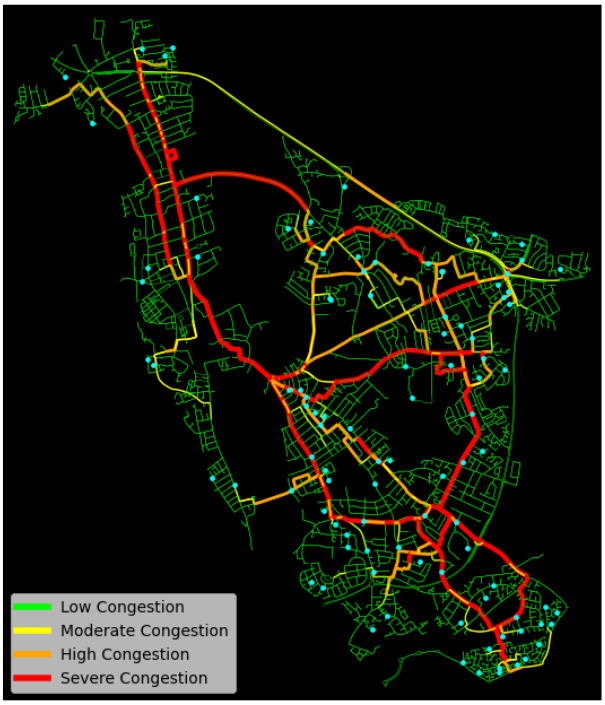

Figure 2.2: Weighted road-network congestion heat-map generated from the Oxford transportation graph after re-routing

The heat-map illustrates how graphs can represent changing traffic conditions, where heavily utilised routes accumulate larger edge weights and appear as regions of increased congestion severity (highlighted in red), whereas underutilised routes remain lower-cost traversal routes.

Road congestion severity was categorised according to relative edge utilisation compared to average network utilisation. Green edges represented underutilised roads, while red edges represented highly congested bottleneck regions with increased travel-time weighting.

The figures demonstrate how congestion weighting alters traffic distribution across the transportation network. This highlights the influence of road design on routing effectiveness.

2.2 Dijkstra's Algorithm

Dijkstra's algorithm remains one of the foundational shortest-path algorithms used within graph optimisation and transportation routing research for shortest-path routing problems and is a common option for researchers attempting to solve pathfinding problems Dijkstra (1959)

The algorithm explores neighbouring nodes iteratively while minimising total cumulative path cost.

Dijkstra's algorithm expands the lowest-cost frontier node while updating neighbouring path costs whenever shorter routes are discovered. The algorithm repeatedly performs edge relaxation where neighbouring path costs are updated if a shorter route is discovered. The relaxation condition is defined as:

$$d(v) > d(u) + w(u, v) \tag{2.1}$$

where:

- $d(v)$ is the current known shortest distance to node v
- $d(u)$ is the shortest distance to node u
- $w(u, v)$ is the edge weight between nodes

If the condition were to hold, then the distance to node v is updated. The computational complexity of Dijkstra's algorithm depends on the data structure used for the priority queue. If using a binary heap implementation, the complexity becomes:

$$O((V + E) \log V) \tag{2.2}$$

where:

- V represents the number of vertices
- E represents the number of edges

Because Dijkstra's algorithm performs uninformed graph traversal, scalability can decrease substantially within large dense transportation networks.

2.3 A* Pathfinding Algorithm

A* extends Dijkstra's algorithm through a heuristic search. Rather than exploring nodes uniformly A* prioritises nodes estimated to be closer to the goal node Hart et al. (1968a).

The mathematical representation of A* is:

$$f(n) = g(n) + h(n) \tag{2.3}$$

where:

- $g(n)$ is the exact path cost from the start node to node n
- $h(n)$ is the heuristic estimate from node n to the goal node

For A* to guarantee optimal shortest-path solutions the heuristic function must be admissible. An admissible heuristic never overestimates the true remaining path cost to the goal node.

The heuristic can be defined as:

$$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{2.4}$$

where:

- x_1 and y_1 represent the coordinates of the current node
- x_2 and y_2 represent the coordinates of the destination node
- $h(n)$ represents the heuristic estimate between the current node and the target node

Dijkstra's algorithm and A* were selected for comparative evaluation due to their widespread use within shortest-path optimisation problems. Dijkstra provides an uninformed optimal-routing baseline, and A* introduces heuristic-guided exploration which is designed to improve scalability and reduce the computational overhead within larger graphs Kokash (2005).

2.4 Traffic Congestion Modelling

Dijkstra's and A* shortest paths are insufficient for large scale transportation modelling as they cannot dynamically evolve with constraints in traffic Hart et al. (1968b). The objective of both algorithms is to minimise path cost between a source and a destination node. Within transportation systems this cost may represent distance, travel time or congestion-adjusted edge weighting. However the shortest-path optimisation alone does not guarantee efficient traffic distribution across an entire network Wardrop (1952).

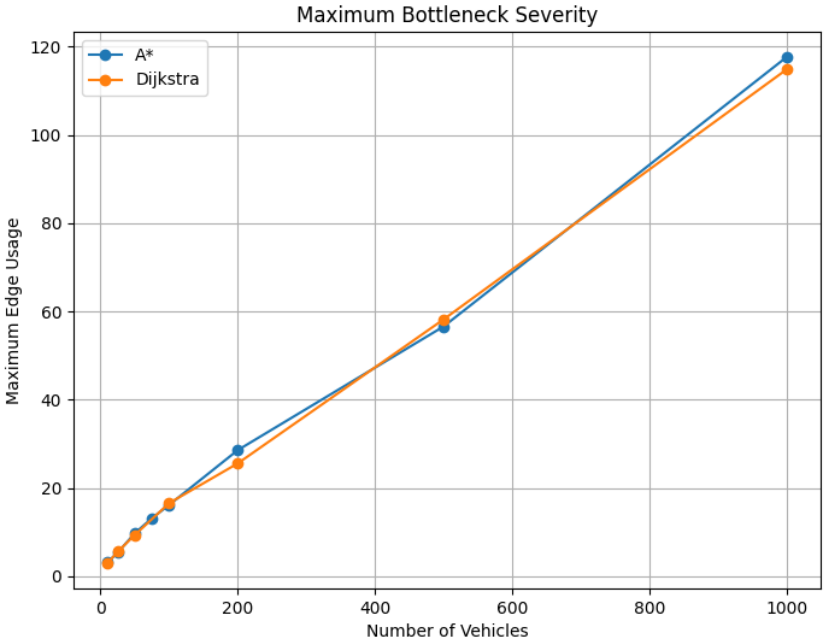

Figure 2.3: Maximum bottleneck severity comparison between Dijkstra and A* under increasing vehicle density

Previous research additionally demonstrates that shortest-path optimisation alone is insufficient for preventing congestion accumulation within dense urban transportation systems.

2.5 Artificial Neural Networks

Biological neurons communicate through interconnected structures such as axons, synapses and dendrites which transmit signals between neurons. Artificial neural networks attempt to model aspects of this behaviour computationally by creating networks of interconnected artificial neurons capable of learning relationships within data Krogh (2008).

ANNs consist of layers of neurons connected through weighted links. Input data passes through the network where weighted sums are calculated and transformed using activation functions to produce an output. During training, the network adjusts these weights using methods such as feed-forward propagation, back-propagation and gradient descent in order to reduce prediction error and improve accuracy.

Artificial neural networks are particularly effective for modelling non-linear traffic relationships which are difficult to represent using deterministic shortest-path optimisation methods. Unlike traditional shortest-path algorithms which primarily optimise local path costs, neural networks are capable of learning broader traffic-flow relationships from observed network conditions.

Within this dissertation, a feed-forward neural network was implemented from first principles without the use of external machine learning frameworks. The network was

trained to predict congestion multipliers using features extracted from the road network including edge utilisation, road length, speed limits and nearby congestion levels.

While recurrent and graph-based neural networks are commonly used within transportation research, a feed-forward neural network was selected within the dissertation due to its simpler architecture, computational feasibility and suitability for supervised regression-based congestion prediction.

2.6 Related Research and Comparative Studies

2.6.1 Graph-Based Traffic Routing Research

Dijkstra's algorithm and A* were both selected due to their widespread use and effectiveness within shortest-path optimisation problems Candra et al. (2020). Dijkstra's algorithm was initially implemented as a baseline routing algorithm due to its shortest-path guarantees and straightforward integration within weighted graph structures generated using OSMnx and NetworkX. The baseline implementation enabled comparative evaluation against admissible heuristic-guided approaches such as A*.

The existing traffic-routing research frequently highlights the limitations of static shortest-path optimisation within highly connected transportation systems. While shortest-path algorithms minimise local traversal cost, they do not optimise global traffic-flow distribution. Several studies therefore investigate edge weighting and route adaptation techniques in order to reduce bottlenecks and improve traffic redistribution.

Research additionally demonstrated that congestion cannot be entirely eliminated within urban environments due to road structure, constraints and limited route diversity between major arterial pathways Calatayud et al. (2021). Congestion frequently accumulates around highways, motorways and highly connected intersections where alternative traversal pathways are limited. These findings aligned closely with the experimental results observed within this dissertation which heavily utilised roads consistently formed persistent bottleneck regions despite re-routing.

A* further extended the routing system through heuristic-guided exploration using Euclidean-distance estimation. Existing research suggests that admissible heuristics can reduce unnecessary graph traversal and improve scalability compared to uninformed shortest-path algorithms Standley (2010). Literature also indicated how heuristic effectiveness remains dependent on the network topology and route diversity. Similar behaviour was observed within the experimental evaluation, where A* demonstrated moderate improvements in traffic distribution and runtime performance, although both algorithms frequently converged toward similar high-connectivity traversal pathways.

2.7 Hopfield Networks

Hopfield Networks were investigated as potential optimisation for traffic routing as they have a strong theoretical foundation with combinatorial optimisation problems. A Hopfield Network is a recurrent neural network consisting of interconnected neurons which the system evolves towards a stable minimum-energy state. Hopfield (1982) The network dynamics are altered through iterative state updates which are designed to minimise a global energy function.

For a network containing n neurons, the energy function can be expressed as:

$$E = -\frac{1}{2}\sum_{i,j}^{n} w_{ij}x_i x_j + \sum_{i}^{n} \theta_i x_i$$

where:

- n represents the total number of neurons in the network,
- w_{ij} denotes the synaptic weight between neuron i and neuron j,
- x_i and x_j represent the states of the neurons,
- and θ_i corresponds to the bias term or threshold associated with neuron i.

Within routing applications, neurons are able to represent road selections, path activations or even traffic state decisions. Whereas weighted connections encode constraints such as congestion penalties, route continuity or invalid transitions.

In regards to traffic routing, the objective can incorporate factors such as:

- Total travel time
- Congestion density
- Road capacity constraints
- Path continuity
- Intersection penalties
- Traffic balancing

By encoding these constraints into the energy formulation, the network can iteratively converge towards a lower-cost route.

Several limitation emerged during evaluation. The primary issue was that Hopfield Networks are static in regards to optimisation systems. Urban traffic environments are highly dynamic and continuously change over time, whereas Hopfield Networks converge to a fixed equilibrium state. Once convergence occurs, adapting to a changing congestion pattern required a repeat in re-computation of the entire network state.

Hopfield Networks are also highly susceptible to convergence towards local minima rather than globally optimal solutions. In larger-scale systems containing many intersections and possible routes, the energy landscape becomes more complex which reduces solution reliability which can result in suboptimal distributions and unstable routing decisions under certain conditions.

Scalability also presented a major concern. A fully connected Hopfield Network requires weight storage which is proportional to the square of the number of neurons. A fully connected Hopfield Network requires weight storage proportional to n^2, where n represents the number of neurons. As the road network size increases, the computational complexity and memory requirement grow significantly which makes real-time deployment difficult for large environments.

Another limitation involved temporal modelling. Hopfield Networks do not naturally represent sequential decision-making or long-term adaptation. Traffic routing requires continuous feedback integration in order to predict future congestion states, this is where an adaptive policy learning based on evolving traffic behaviour. Reinforcement learning approaches were suited better for these requirements as they optimise actions through interaction with the environment over time rather than converging to a single static energy minimum.

Hopfield Networks were therefore not selected as the primary optimisation architecture. Investigating the model provided valuable theoretical insights into energy-based optimisations and assisted in establishing the design rationale for adopting adaptive reinforcement learning methods within the final optimisation system.

2.8 Feed-forward Neural Networks

Feed-forward neural networks were investigated for congestion prediction due to their ability to model non-linear relationships between transportation variables. Arulampalam and Bouzerdoum (2003).

A standard feed-forward network can consist of:

- an input layer
- one or more hidden layers
- an output layer

Each neuron receives weighted inputs from neurons in the previous layer and computes and activation value. For neuron j, the weighted input can be expressed as:

$$z_j = \sum_{i=1}^{n} w_{ij} x_i + b_j$$

where:

- x_i represents the input features,

- w_{ij} denotes the weight connecting neuron i to neuron j,
- b_j represents the bias term,
- z_j is the weighted sum input to neuron j,
- and n corresponds to the total number of input connections.

The activation output of neuron j can then be expressed as:

$$a_j = f(z_j)$$

where:

- a_j represents the neuron activation output,
- $f(z_j)$ represents the activation function.

Goodfellow et al. (2016).

In the context of traffic systems, an input variable may include:

- vehicle density,
- average traffic speed,
- intersection wait times,
- road occupancy,
- traffic signal states,
- historical congestion measurements

The network output can then be used to predict future congestion levels and estimate route costs.

The implemented feed-forward neural network lacked temporal memory and therefore could not model evolving traffic dependencies over time as effectively as recurrent or graph-based architectures.

Although more advanced architectures may provide stronger temporal modelling capabilities, a feed-forward neural network was selected due to its implementation simplicity and suitability for supervised congestion prediction within the scope of this dissertation. Goodfellow et al. (2016)

2.8.1 Neural-Network-Based Traffic Prediction Research

Existing transportation research increasingly applies machine-learning approaches to congestion prediction and intelligent traffic optimisation. Neural-network-based systems are particularly effective due to their ability to model highly non-linear traffic relationships that are difficult to represent using traditional routing systems.

Previous studies have used feed-forward neural networks to predict traffic-flow behaviour using variables such as vehicle density, road occupancy and traffic speed. However, many of these approaches struggle when congestion becomes highly dynamic or unpredictable. Vlahogianni et al. (2004). These approaches can then improve shortest-path optimisation by allowing traffic conditions to influence future route-selection.

More advanced architectures including recurrent neural networks (RNNs), long short-term memory networks (LSTMs), graph neural networks (GNNs) and reinforcement learning systems have also been investigated within transportation research. These architectures are capable of modelling traffic dependencies and large-scale graph relationships more effectively than standard feed-forward networks.

Although these architectures often achieve stronger predictive performance, they also introduce substantially greater computational complexity and implementation overhead Wu et al. (2021a). Graph neural networks in particular require significantly larger datasets and more computational resources due to repeated message-passing operations across the graph Wu et al. (2021b).

For this dissertation, a custom feed-forward neural network implemented entirely from first principles was selected due to its computational simplicity and suitability for supervised congestion regression.

Chapter 3

METHODOLOGY

3.1 Research Approach

The project adopted an experimental simulation-based research methodology in order to investigate adaptive traffic routing within weighted transportation networks. A simulation approach was selected because it enabled controlled evaluation of routing behaviour under varying traffic conditions while also allowing direct comparison between routing algorithms and congestion prediction.

The research combined both classical graph-theoretic optimisation and machine-learning-based prediction within a transportation framework. Dijkstra's algorithm and A* were comparatively evaluated alongside adaptive congestion weighting and a custom feed-forward neural network used for congestion prediction.

A real-world transportation network extracted from OpenStreetMap was used throughout experimentation in order to improve realism compared to synthetic graphs. Simulation-based experimentation additionally enabled repeatable testing under controlled traffic conditions while avoiding the complexity and accessibility constraints associated with real-world traffic infrastructure. Vlahogianni et al. (2014b)

The methodology focused on how edge weighting and learned congestion prediction affected routing behaviour, bottleneck formation and traffic distribution as traffic volumes increased.

3.2 Graph-Theoretic Properties of Transportation Networks

The network can be represented mathematically as a weighted directed graph:

$$G = (V, E) \tag{3.1}$$

where:

- V represents the set of graph vertices corresponding to road intersections,
- E represents the set of directed edges corresponding to road segments.

Each edge within the graph is then assigned with a positive traversal cost through the weighting function:

$$w : E \to \mathbb{R}^+ \tag{3.2}$$

where $w(e)$ represents the travel-time cost associated with the edge e.

In regards to transportation systems, edge weights can represent multiple traversal constraints including:

- road length,
- estimated travel time,
- congestion severity,
- speed-limit restrictions,
- dynamic traffic conditions.

Real-world networks typically create sparse, complex graph structures compared to fully connected graphs. This relationship can be expressed as:

$$|E| \ll |V|^2 \tag{3.3}$$

where:

- $|E|$ represents the total number of edges,
- $|V|^2$ represents the maximum number of edges within a fully connected graph.

The sparsity occurs because road systems are constrained and intersections cannot connect directly to all other intersections within the network. Sparse graphs can significantly influences shortest-path complexity with congestion formation and routing scalability.

3.3 Network Optimisation Formulation

Transportation routing within a network can be formed as a constrained optimisation problem. Instead of considering individual shortest-path calculations independently, the transportation system can be seen globally as a flow-distribution problem across a weighted directed graph.

Let the network be represented as:

$$G = (V, E) \tag{3.4}$$

where V denotes the set of intersections and E denotes the set of directed road segments.

Each edge $e \in E$ holds an associated congestion-adjusted traversal cost w_e alongside a traffic flow value f_e representing the number of vehicles traversing that edge during simulation.

The global routing objective can then be expressed as the minimisation of total network traversal cost:

$$J = \sum_{e \in E} w_e f_e \tag{3.5}$$

where:

- J represents the total transportation cost across the network,
- w_e represents the congestion-adjusted edge weight,
- f_e represents traffic flow through edge e.

As congestion increases, edge costs increase proportionally which encourages traffic redistribution to other viable routes where feasible.

The optimisation process is constrained by road-capacity limitations. For each edge:

$$f_e \leq C_e \tag{3.6}$$

where:

- f_e represents observed traffic flow,
- C_e represents estimated road capacity.

When traffic flow approaches or exceeds the edge capacity, congestion multipliers increase which causes traversal costs to grow non-linearly throughout the graph.

The routing system therefore attempts to minimise cumulative transportation cost while also adapting to dynamically evolving congestion conditions.

3.4 Dynamic Congestion Evolution

Re-routing was implemented through dynamically evolving edge weights. As vehicle utilisation increased across specific road segments, traversal costs increased proportionally to encourage alternative traversal pathways.

The edge-weight update mechanism can be expressed as:

$$w_e(t+1) = w_e(t)\,(1 + \alpha U_e(t)) \tag{3.7}$$

where:

- $w_e(t)$ represents the edge weight at time t,

- $U_e(t)$ represents edge utilisation,
- α represents the congestion sensitivity parameter.

Edge utilisation was calculated using:

$$U_e = \frac{V_e}{C_e} \tag{3.8}$$

where:

- V_e represents the number of vehicles traversing edge e,
- C_e represents the estimated edge capacity.

As utilisation increases, larger travel-time penalties were applied to congested roads. This transformed the network from a static shortest-path problem into a dynamic optimisation system.

3.5 Betweenness Centrality and Bottleneck Formation

There were many concurrent bottlenecks that were forming in highly utilised zones as shown in the evaluation chapter. In graph theory, this behaviour can be analysed using betweenness centrality.

Betweenness centrality for node v can be defined as:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \tag{3.9}$$

where:

- σ_{st} represents the total number of shortest paths between nodes s and t,
- $\sigma_{st}(v)$ represents the number of shortest paths passing through node v.

Nodes with high betweenness centrality attracted larger traffic volumes as many shortest-path solutions had converged through these intersections. This explains why several roads always formed persistent congestion bottlenecks even though vehicles were re-routed.

3.6 Heuristic Analysis of A* Routing

A* routing extends off Dijkstra's algorithm by incorporating heuristic-guided graph traversal.

The heuristic function must remain admissible in order to guarantee optimal shortest-path solutions:

$$h(n) \leq h^*(n) \tag{3.10}$$

where:

- $h(n)$ represents the estimated remaining path cost,
- $h^*(n)$ represents the true shortest remaining path cost.

Euclidean distance was selected as the heuristic as the straight-line distance does not overestimate actual road distance. The heuristic then remained admissible throughout route computation.

The heuristic additionally satisfies the consistency condition:

$$h(n) \leq c(n, n') + h(n') \tag{3.11}$$

where:

- $c(n, n')$ represents the traversal cost between neighbouring nodes.

Consistency ensures that estimated path costs remain monotonic during graph traversal which improves the search efficiency and can guarantee optimality.

3.7 Traffic Flow Theory

Traffic-flow can be analysed mathematically using relationships between traffic density, flow and velocity.

Traffic flow can be represented as:

$$q = kv \tag{3.12}$$

where:

- q represents traffic flow,
- k represents vehicle density,
- v represents average vehicle velocity.

Greenshields (1935)

As vehicle density increases throughout the network, average velocity decreases due to congestion accumulation and increased interaction between each vehicle.

3.8 Gradient Descent and Back-Propagation

The feed-forward neural network was trained using gradient descent and back-propagation.

Weight updates were computed using partial derivatives of the loss function with each network parameter.

For the output layer, gradients can be expressed using the chain rule:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial Z_2} \frac{\partial Z_2}{\partial W_2} \tag{3.13}$$

where:

- L represents the loss function,
- $\hat{y}$ represents the predicted output,
- W_2 represents the output-layer weights.

The gradient values were then used to iteratively update network parameters through gradient descent optimisation:

$$W := W - \eta \frac{\partial L}{\partial W} \tag{3.14}$$

where:

- η represents the learning rate.

This optimisation gradually reduced prediction error throughout training and also enabled the neural network to learn congestion relationships effectively from the dataset provided.

Back-propagation within hidden layers was computed recursively using the chain rule. For hidden layer l, the error term can be expressed as:

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot f'(z^{(l)}) \tag{3.15}$$

where:

- $\delta^{(l)}$ represents the error vector for layer l,
- $W^{(l+1)}$ represents the weight matrix of the subsequent layer,
- $f'(z^{(l)})$ represents the derivative of the activation function,
- $\odot$ denotes element-wise multiplication.

The ReLU activation function was selected because its derivative remains computationally simple for positive activation values:

$$f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases} \tag{3.16}$$

This helped reduce vanishing-gradient behaviour during optimisation and improved training stability.

3.9 Bias-Variance Behaviour

Prediction performance can be analysed using the bias-variance decomposition:

$$\mathbb{E}[(y - \hat{f}(x))^2] = Bias^2 + Variance + Noise \tag{3.17}$$

where:

- $Bias^2$ represents systematic prediction error,
- $Variance$ represents model sensitivity to training data,
- $Noise$ represents irreducible uncertainty within the dataset.

The implemented feed-forward neural network showed relatively high bias under severe congestion conditions as the provided training data didn't have sufficient data generation that could vary the results.

This biased training set caused the neural-network to under-fit during prediction when it came to larger congestion multipliers.

3.10 Scalability Analysis

The computational complexity of the simulation increased significantly as vehicle density became larger.

For N simulated vehicles, total routing complexity can be approximated as:

$$T(N) \approx N \cdot O((V + E) \log V) \tag{3.18}$$

where:

- N represents the number of vehicles,
- V represents graph vertices,
- E represents graph edges.

Additional computational overhead was introduced by:

- congestion recalculation,

- adaptive re-routing,
- graph traversal,
- neural-network prediction.

As traffic density increased, repeated shortest-path calculations and congestion updates produced a substantial increase in runtime which limited scalability for the larger environments.

3.11 Experimental Environment

- OSMnx for road-network extraction from OpenStreetMap
- NetworkX for graph representation and shortest-path routing
- NumPy for numerical computation
- Pandas for dataset management
- Matplotlib for graph visualisation and congestion heat-maps

The network was represented using a weighted `MultiDiGraph` structure where intersections corresponded to nodes and roads corresponded to directed weighted edges.

All simulations were executed locally using vehicle generation across the Oxford road network extracted through OpenStreetMap.

3.12 Road-Network Data Collection

Real-world road-network data was extracted from OpenStreetMap using the OSMnx library. Oxford was selected as the primary experimental region due to its mixture of dense urban roads and complex intersections because it provided suitable conditions for congestion modelling evaluation.

The road network was generated using geographic coordinate queries and converted into a weighted directed graph compatible with NetworkX. Nodes represented road intersections while edges represented traversable road segments containing metadata such as:

- road length,
- highway classification,
- geographic coordinates,
- travel-time weighting

3.13 Vehicle Simulation Methodology

Vehicles were simulated across the transportation graph using randomly generated origin-destination node pairs selected from valid connected graph regions.

Each simulated vehicle contained:

- an origin node,
- a destination node,
- a calculated shortest-path route,
- a selected routing algorithm

Vehicle routes were generated using either Dijkstra's algorithm or the A* pathfinding algorithm depending on configuration.

Traffic density was controlled by varying the number of active vehicles. Experimental vehicle volumes ranged from 10 to 1000 vehicles in order to evaluate scalability and adaptive re-routing decisions under high volumes of vehicles.

3.14 Congestion Modelling Strategy

Traffic congestion was modelled using edge-utilisation weighting throughout the graph.

After all vehicle routes had been generated, the edge traversal frequencies were measured by counting how often each road segment appeared across all active routes. Congestion multipliers were then applied to heavily utilised edges in order to increase their effective travel-time cost.

The updated edge weighting mechanism allowed the graph to evolve based on surrounding traffic conditions. Vehicles recalculated routes using the updated graph which enabled adaptive re-routing behaviour.

Congestion severity was categorised according to relative edge utilisation compared against average network utilisation. Higher utilisation values produced larger travel-time multipliers and increased congestion severity within heat-map visualisations.

This transformed the system from a static shortest-path problem into a dynamic weighted-routing problem.

Rather than training the neural network using predefined heuristic congestion multipliers, the final implementation exported observed congestion ratios generated during adaptive simulation. This allowed the model to learn simulation-derived congestion behaviour directly from routing dynamics.

3.15 Neural Network Training Methodology

A custom feed-forward neural network was implemented entirely from first principles without external machine-learning frameworks in order to predict congestion multipliers from traffic-related features.

The network received several congestion-related input features including:

- edge utilisation,
- road length,
- speed limit,
- nearby congestion levels,
- encoded road classifications

The network architecture consisted of:

- an input layer,
- two hidden layers,
- a single regression output neuron

Training was performed using forward propagation, back-propagation and gradient descent optimisation with Mean Squared Error (MSE) used as the loss function.

3.15.1 Dataset Preparation

Feature standardisation was applied in order to stabilise gradient descent and reduce sensitivity to feature magnitude differences between traffic variables.

Input features were standardised using z-score normalisation:

$$X_{scaled} = \frac{X - \mu}{\sigma} \tag{3.19}$$

where:

- X represents the original feature value
- μ represents the feature mean
- σ represents the feature standard deviation

The scaler parameters were saved after training and reused during inference in order to ensure consistent feature transformation between training and prediction stages.

The congestion target values were additionally normalised into the range $[0, 1]$ before training:

$$y_{scaled} = \frac{y - y_{min}}{y_{max} - y_{min}} \tag{3.20}$$

Target normalisation improved optimisation stability and reduced prediction variance during neural-network training.

3.16 Evaluation Methodology

System performance was evaluated using both routing-performance metrics and neural-network regression metrics.

Routing evaluation measured:

- runtime complexity,
- average edge utilisation,
- maximum bottleneck severity,
- average travel cost,
- search-space exploration,
- adaptive re-routing frequency

The neural network was evaluated using standard regression metrics such as:

- Mean Squared Error (MSE),
- Root Mean Squared Error (RMSE),
- Mean Absolute Error (MAE)

Comparative experiments were conducted across increasing traffic densities in order to analyse the scalability and congestion re-routing under more demanding transportation conditions.

Chapter 4

SYSTEM DESIGN

4.1 Road Network Generation

Road networks were generated using real-world geographical data which was extracted from OpenStreetMap through the OSMnx Python library. Realistic transportation modelling required accurate representation of road topology and intersection connectivity which was provided by the library. Synthetic graph structures were considered insufficient for the dissertation due to failure with representing irregular structure and traversal constraints with urban transportation systems. Boeing (2017)

OSMnx was selected due to its ability to automatically construct transportation graphs directly from OpenStreetMap data while maintaining compatibility with NetworkX. This library enabled extraction of road layouts, intersections, road classifications, speed-limits and geographical coordinates through functions such as:

```
ox.graph_from_point()
```

and:

```
ox.graph_from_place()
```

The extracted road network was represented as a weighted directed `MultiDiGraph` where intersections corresponded to graph nodes and roads corresponded to directed edges. A `MultiDiGraph` structure was necessary as transportation systems contain multiple parallel road segments along with lane variation and one-way routing structures between the same pair of intersections.

Each edge stored geographical and transportation metadata including:

- road length,
- road classification,
- edge geometry,
- travel-time weighting,

- geographical coordinates

These attributes were used for shortest-path routing, congestion weighting, heuristic calculations and heat-map visualisations.

Location generation performed using string-based geographical queries produced an approximate coordinate estimation which occasionally generated invalid routing behaviour and inaccurate node placement. In order to improve routing precision, the implementation was altered to use direct latitude and longitude coordinate inputs.

Vehicle routing required mapping arbitrary geographic coordinates onto valid graph nodes. This was achieved by using "nearest-node" search functions provided within OSMnx. In development, several routing failures occurred due to the disconnected graph regions and isolated road components within the network which produced the following exception:

```
networkx.exception.NetworkXNoPath
```

The issue occurred when randomly generating origin to destination pairs which did not possess a valid path within the graph structure. To fix this issue, route validation and exception-handling was implemented to ensure that only connected node pairs were accepted during vehicle creation.

Geographical coordinates extracted from OpenStreetMap enabled heuristic distance estimation for the A* pathfinding algorithm. Euclidean-distance heuristics were calculated from the node latitude and longitude values, in order to estimate a straight-line distance between the current node and its destination node. This provided an admissible heuristic approximation while remaining computationally efficient for larger scale operations.

4.2 System Architecture

- 1. Generate transportation graph $G(V,E)$
- 2. Generate valid vehicles
- 3. Compute shortest-path routes
- 4. Calculate edge utilisation
- 5. Apply congestion multipliers
- 6. Recompute weighted shortest paths
- 7. Evaluate congestion metrics
- 8. Train neural network
- 9. Predict congestion multipliers
- 10. Reapply adaptive weighting

4.3 Vehicle Simulation

Vehicles were simulated across the network in order to model realistic congestion behaviour within the Oxford road graph.

Each vehicle contained:

- A start node
- A destination node
- A current position
- A calculated route
- A selected routing algorithm

Vehicles were generated using valid graph nodes extracted directly from the road network rather than random geographic coordinates. Initially, random coordinates were considered; however, this produced invalid vehicle placements because some coordinates did not correspond to valid roads or connected graph regions.

To resolve this issue, vehicle generation was instead constrained to existing graph nodes obtained from the network itself. This ensured that all generated vehicles were positioned on valid traversable road segments.

For each vehicle, a random origin and destination node were selected from the graph. The routing algorithm then calculated the shortest available route between both nodes using either Dijkstra's algorithm or A*.

The vehicle generation process followed the structure:

```
valid_route = False

while not valid_route:

    generate random nodes

    try:

        create vehicle

        add vehicle

        valid_route = True

    except nx.NetworkXNoPath:

        retry
```

Once all vehicles had been generated, edge utilisation statistics were calculated by counting how frequently each road segment appeared across all vehicle routes. These edge frequencies were later used to generate congestion weightings and congestion heat-map visualisations.

The simulation was executed using multiple traffic densities ranging from small vehicle counts up to 1000 simultaneously simulated vehicles. Higher traffic loads allowed the system to evaluate congestion growth, bottleneck severity, routing scalability, and adaptive re-routing behaviour under larger transportation loads.

4.4 Congestion Weighting

Each road segment stores edge usage counts.

As vehicle counts increase, travel-time multipliers increase proportionally.

Usage Ratio	Traffic Level	Multiplier
< 0.5	Very Low Traffic	1.0x
0.5 – 1.0	Normal Traffic	1.1x
1.0 – 1.5	Moderate Congestion	1.4x
1.5 – 2.0	Heavy Congestion	1.8x
> 2.0	Severe Congestion	2.5x

Table 4.1: Congestion weighting model

Congestion weighting transformed the transportation graph from a static shortest-path optimisation problem into a dynamically evolving weighted-routing system. Initially, all roads possessed baseline travel-time costs derived from road length and speed-limit attributes that were taken directly from OpenStreetMap. However, as vehicle density increased, heavily utilised roads accumulated larger congestion multipliers which increased traversal cost across the corresponding graph edges.

The updated travel-time weighting can be represented as:

$$T_{new} = T_{base} \times M_c \tag{4.1}$$

where:

- T_{new} represents the updated travel-time weight
- T_{base} represents the baseline edge travel time
- M_c represents the congestion multiplier

This adaptive weighting mechanism led vehicles to avoid heavily congested roads during re-routing phases by increasing the effective traversal cost of overloaded edges.

Congestion severity was determined using relative edge utilisation compared against average graph utilisation. Edge utilisation can be defined as:

$$U_e = \frac{V_e}{C_e} \tag{4.2}$$

where:

- U_e represents edge utilisation
- V_e represents the number of vehicles traversing the edge
- C_e represents the estimated edge capacity

As edge utilisation increased beyond average network utilisation, larger congestion multipliers were applied. This enabled the transportation graph to evolve dynamically according to surrounding traffic conditions.

4.5 Heat-map Visualisation

Traffic heat-maps were implemented to visualise congestion distribution.

Road colours represent congestion severity:

- Green = low congestion
- Yellow = moderate congestion
- Orange = heavy congestion
- Red = severe congestion

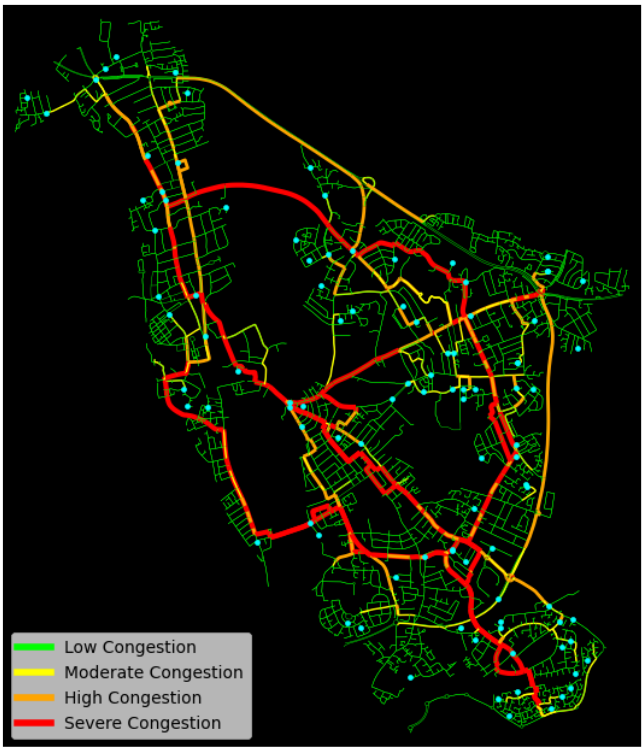

Figure 4.1: Congestion heat-map illustrating edge utilisation across the Oxford road network for 100 simulated vehicles

Edge colours dynamically change depending on the edge use where higher edge usage indicate a red colour and lower edge usage range from green to orange.

Chapter 5

IMPLEMENTATION

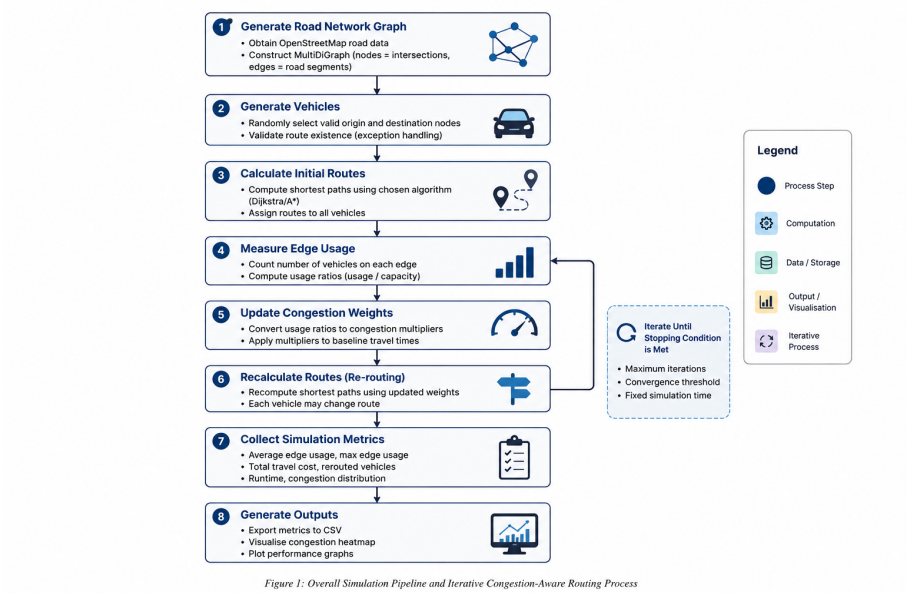

Figure 5.1: Overall adaptive congestion-aware routing and simulation pipeline

Figure 5.1 illustrates the overall architecture and execution pipeline of the proposed adaptive traffic-routing system. The simulation begins by generating a weighted transportation graph from OpenStreetMap data before dynamically generating vehicles with valid start to destination pairs. Initial shortest-path routes are calculated using either Dijkstra or A* pathfinding before congestion metrics are measured across graph edges. Congestion-aware travel-time weights are then applied to heavily utilised roads, after which adaptive re-routing is performed. Finally, simulation metrics and congestion visualisations are generated for evaluation.

5.1 Technology Stack

- Python

- NetworkX
- OSMnx
- NumPy
- Pandas
- Matplotlib

5.2 Routing Algorithms

Shortest-path routing in the graph was implemented using both Dijkstra's algorithm and the A* pathfinding algorithm through the NetworkX library.

Routing was performed on a weighted `MultiDiGraph` generated from OpenStreetMap data. Intersections were represented as nodes, while road segments were represented as directed weighted edges.

For each generated vehicle, the system selected a routing algorithm depending on the simulation configuration. Dijkstra's algorithm calculated routes using cumulative travel-time cost without heuristic guidance, and A* additionally incorporated Euclidean-distance heuristics in order to guide graph traversal toward the destination node more efficiently Cormen et al. (2009).

Dijkstra routing was implemented using:

```
nx.shortest_path()
```

while A* routing used:

```
nx.astar_path()
```

The Euclidean heuristic used by A* was calculated from node latitude and longitude coordinates which came from the OpenStreetMap library:

$$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{5.1}$$

where:

- x_1 and y_1 represent the coordinates of the current node
- x_2 and y_2 represent the coordinates of the destination node

Vehicles followed static shortest-path routes based only on baseline travel-time costs. Although, after congestion metrics were calculated, edge weights were dynamically updated using congestion multipliers. Vehicles were then re-routed using the updated graph which allowed the system to simulate routing.

This implementation allowed comparative evaluation between uninformed shortest-path optimisation and heuristic-guided routing under dynamically changing traffic conditions.

5.3 Congestion Re-routing

In the start vehicles traversed the transportation graph using static shortest-path algorithm route which was generated from the baseline travel-time weights. However, this approach failed to account for dynamically changing congestion conditions caused by increasing edge utilisation across the network.

In order to address this limitation, congestion-aware route adaptation was implemented. After all initial vehicle routes were generated, edge utilisation statistics were calculated by measuring how frequently each road segment was traversed during the simulation. Congestion multipliers were then applied to more congested edges to increase their effective travel-time cost.

Once updated edge weights had been applied, vehicles recalculated their routes using the modified weighted graph. This enabled the routing algorithms to avoid congested road segments where alternative pathways were available.

The re-routing process followed three primary stages:

1. Generate initial shortest-path routes
2. Calculate global congestion metrics and update edge weights
3. Recalculate vehicle routes using congestion-adjusted travel times

The issue was resolved by separating route generation, congestion calculation, and re-routing into distinct global simulation stages. This reduced unnecessary re-computation and improved scalability for larger simulations involving hundreds of active vehicles.

5.4 Route Traversal Correction

During development, an implementation error was identified within the edge traversal logic used for congestion calculation.

Edge extraction from vehicle routes was performed using:

```
edge = (route[1], route[i+1])
```

This implementation incorrectly referenced the second route node rather than the current traversal node during iteration. This corrupted edge utilisation metrics as multiple road segments were mapped to incorrect edge pairs during congestion analysis.

The implementation was corrected using:

```
edge = (route[i], route[i+1])
```

This modification ensured that edge traversal correctly represented sequential node transitions.

Congestion calculations accurately reflected real traversal frequencies across the graph which improved the reliability of congestion heat-maps, edge-weight updates, and re-routing decisions.

5.5 System Architecture

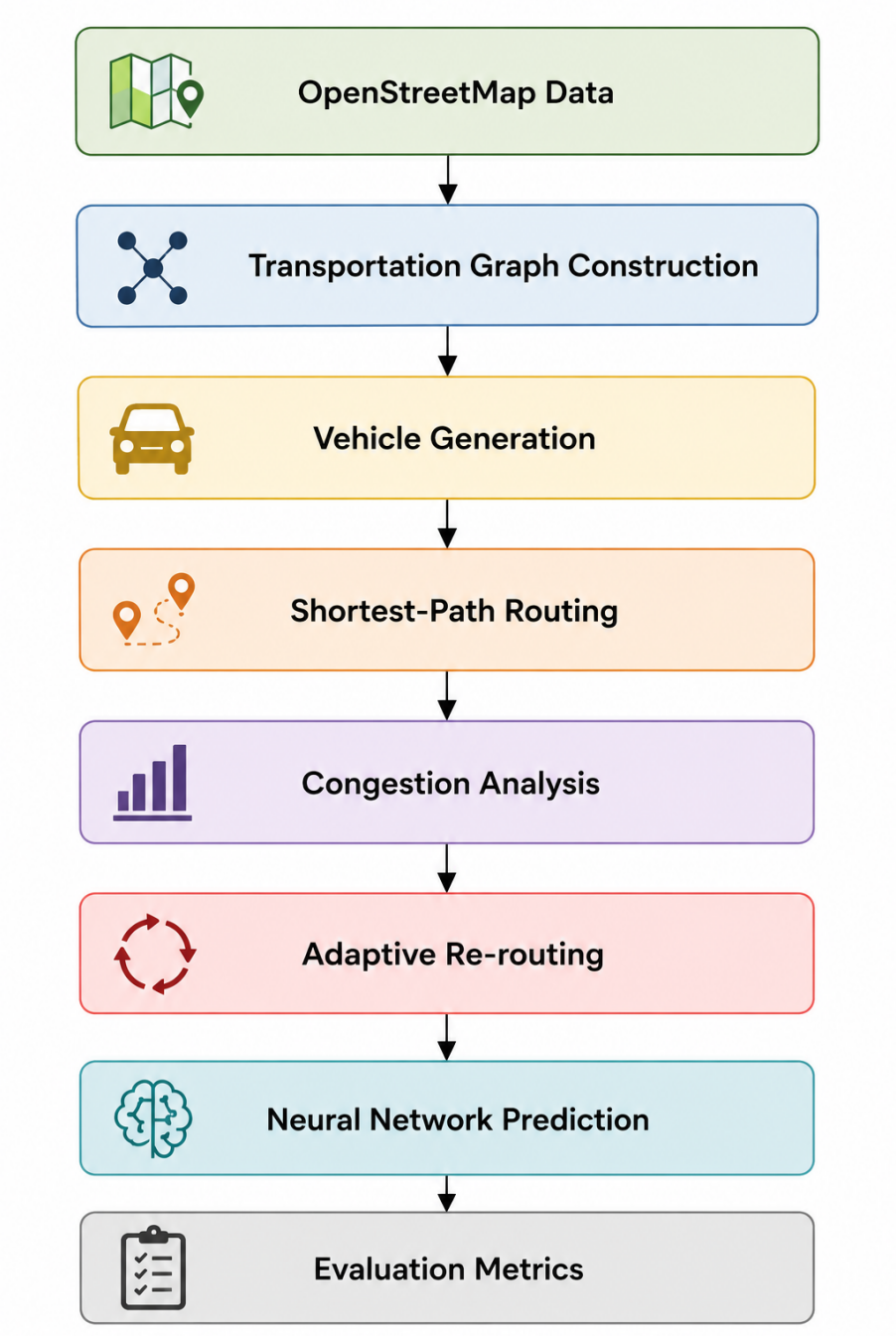

Figure 5.2: Overall system architecture and workflow

Figure 5.2 illustrates the overall workflow of the transportation-routing and congestion-optimisation framework developed throughout this dissertation. The system integrates

graph construction, routing, adaptive re-routing, and neural-network-based congestion prediction within a unified simulation pipeline.

5.6 Implementation Refinements and Debugging

Throughout development, several implementation issues were identified and corrected in order to improve routing accuracy, modelling reliability, and computational scalability.

5.6.1 Disconnected Graph Components

During vehicle generation, randomly selected origin and destination nodes occasionally belonged to disconnected graph regions within the OpenStreetMap transportation network. This produced the exception:

```
networkx.exception.NetworkXNoPath
```

Some generated routes were unreachable within the directed transportation graph due to disconnected road segments. This demonstrates how road topology affects traffic distribution under constrained conditions. Vehicles are forced towards similar highly utilised roads and the results show that the road network topology is a major factor in the effectiveness of adaptive re-routing.

To resolve this issue, route validation and exception handling were introduced:

```
valid_route = False

while not valid_route:

    try:

        vehicle = Vehicle(...)

        valid_route = True

    except nx.NetworkXNoPath:

        retry
```

This ensured that only valid connected routes within the network graph were included in the simulation while preventing runtime termination during large-scale vehicle generation.

5.6.2 Exponential Re-routing Complexity

A computational scalability issue occurred during early re-routing development where route recalculation logic was repeatedly executed within nested vehicle loops.

This caused all vehicle routes to be recalculated during each congestion update cycle which produced a severe computational overhead as the number of vehicles increased. Larger simulations became computationally infeasible due to exponential growth in re-routing operations.

To reduce this overhead, the simulation was restructured into three global stages:

1. Generate all initial routes
2. Calculate global congestion metrics
3. Recalculate routes once using updated edge weights

This reduced repeated computations and significantly improved scalability during larger traffic simulations involving hundreds of active vehicles.

5.6.3 Route Traversal Indexing Error

During congestion analysis, an indexing error was identified within the edge traversal logic:

```
edge = (route[1], route[i+1])
```

The implementation incorrectly referenced the second route node rather than the current traversal node. This caused incorrect edge-pair generation during congestion calculations and produced invalid bottleneck measurements across the graph.

The issue was corrected using:

```
edge = (route[i], route[i+1])
```

Following correction, edge utilisation statistics accurately reflected sequential node traversal throughout vehicle routes which improved congestion weighting reliability and heat-map generation.

5.6.4 Congestion Weight Instability

A stability issue emerged when congestion multipliers were repeatedly applied to already-modified travel-time weights during multiple re-routing cycles.

This produced exponential edge-weight growth and unrealistic traversal costs throughout the transportation graph.

To prevent instability, a baseline travel-time attribute was introduced into the routing system:

```
base_travel_time
```

Congestion multipliers were then always applied relative to the baseline travel-time value rather than recursively modifying previously updated weights.

This stabilised congestion weighting behaviour and prevented uncontrolled recursive edge-cost amplification.

5.6.5 MultiDiGraph Edge Updating

The transportation network was represented using a `MultiDiGraph` structure in order to support multiple directed edges between the same pair of nodes.

Initial congestion updates modified only a single edge key:

```
first_key = list(edge_data.keys())[0]
```

This caused inconsistent congestion propagation because parallel road segments remained unaffected during congestion updates.

The issue was resolved by iterating through all edge keys:

```
for key in edge_data:
```

This ensured that all parallel directed road segments correctly received congestion-adjusted travel-time updates during re-routing.

5.6.6 Visualisation Synchronisation Issue

During evaluation, congestion heat-map visualisations appeared unchanged between static routing and re-routing simulations despite route recalculations occurring internally.

This occurred because edge utilisation statistics and plotting data were not being refreshed after re-routing. This meant that visual outputs continued displaying outdated congestion information from the original vehicle routes.

The problem was resolved by resetting edge utilisation statistics and regenerating congestion plots after route recalculation.

This ensured that visual outputs accurately reflected post-rerouting traffic-flow behaviour

5.6.7 Neural Network Input Dimension Errors

During integration of the feed-forward neural network, shape mismatch errors occurred during forward propagation:

```
ValueError: shapes not aligned
```

The issue occurred because the neural network expected five input features while the provided feature vector contained inconsistencies in dimensional structure.

The implementation was corrected by restructuring the input format:

```
sample = np.array([features])
```

Additional feature modifications ensured that all required congestion features were correctly aligned with the neural-network architecture.

After fixing, forward propagation and congestion prediction operated successfully across the generated dataset.

5.7 Neural Network Architecture

The neural network predicts congestion multipliers using:

- Edge usage
- Road length
- Speed limit
- Nearby congestion
- Encoded road type

5.8 Forward Propagation

Forward propagation computes:

$$Z_1 = XW_1 + b_1 \tag{5.2}$$

5.9 Loss Function

The Mean Squared Error loss function is:

$$L = \frac{1}{n}\sum(y - \hat{y})^2 \tag{5.3}$$

5.10 Gradient Descent

Weights are updated using:

$$W := W - \eta\frac{\partial L}{\partial W} \tag{5.4}$$

The implementation phase successfully integrated graph-based shortest-path routing, adaptive congestion weighting, congestion visualisation, and feed-forward neural-network

prediction within a unified transportation simulation framework. Multiple implementation refinements were required throughout development to improve routing reliability, congestion accuracy, and computational scalability. The completed system subsequently enabled large-scale experimental evaluation under dynamically evolving traffic conditions.

Chapter 6

EXPERIMENTAL EVALUATION

This chapter evaluates the performance of the proposed congestion-aware routing framework under increasing traffic densities. Comparative experiments were conducted using both Dijkstra's algorithm and A* routing in order to analyse routing scalability, congestion behaviour, adaptive re-routing effectiveness, and neural-network congestion prediction performance across the Oxford transportation network.

6.1 Evaluation Metrics

System performance was evaluated using both routing-performance metrics and neural-network regression metrics in order to analyse transportation efficiency, congestion behaviour, routing scalability, and prediction accuracy.

The neural network was evaluated using standard regression metrics including:

- Runtime complexity
- Average edge utilisation
- Maximum bottleneck severity
- Total travel cost
- Average travel cost
- Search-space exploration
- Adaptive re-routing frequency

These metrics were selected to evaluate how effectively the routing algorithms distributed traffic throughout the network under dynamically evolving congestion conditions.

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

- Mean Absolute Error (MAE)
- Coefficient of Determination (R^2)

These metrics measured prediction accuracy, convergence behaviour, and the ability of the neural network to model non-linear congestion relationships throughout the generated traffic dataset.

6.2 Routing Evaluation

Routing experiments were conducted under progressively increasing traffic densities in order to evaluate routing scalability under varying loads.

The simulations used the following vehicle counts:

- 10 vehicles
- 25 vehicles
- 50 vehicles
- 75 vehicles
- 100 vehicles
- 200 vehicles
- 500 vehicles
- 1000 vehicles

Lower vehicle densities were used to analyse baseline routing behaviour under relatively unconstrained traffic conditions whereas larger simulations enabled investigation of congestion accumulation, bottleneck formation, search-space complexity, and adaptive re-routing effectiveness under heavier network utilisation.

Increasing vehicle density additionally allowed comparative evaluation of Dijkstra's algorithm and A* under progressively constrained transportation conditions where congestion weighting increasingly influenced route selection.

6.3 Neural Network Hyper-parameters

The feed-forward neural network was configured using a lightweight architecture in order to balance computational efficiency with the ability to model congestion relationships throughout the network.

The network architecture consisted of:

- An input layer containing five traffic-related features

- A first hidden layer containing 16 neurons
- A second hidden layer containing 8 neurons
- A single output neuron used to predict congestion multipliers

The output layer used a linear activation function because congestion prediction was treated as a regression problem rather than classification.

The neural network was trained using gradient descent optimisation with Mean Squared Error (MSE) as the primary loss function.

ReLU activation functions were used within hidden layers due to their computational simplicity and stable gradient behaviour during optimisation.

Training utilised the following hyper-parameters:

- Input features: 5
- Hidden-layer structure: 16-8
- Output features: 1
- Learning rate: 0.0001
- Number of training epochs: 1000

Feature standardisation and target normalisation were additionally applied in order to improve optimisation stability and reduce sensitivity to differing feature magnitudes during training.

This architecture was selected due to the moderate size of the generated congestion dataset and the computational overhead associated with retraining during experimentation.

The hidden-layer sizes were selected empirically in order to provide sufficient representational capacity for modelling non-linear congestion relationships while avoiding excessive over-parameterisation relative to the dataset size.

Experimental evaluation indicated that increasing network depth produced limited improvements in prediction performance while significantly increasing computational cost and training instability.

6.4 Runtime Complexity Evaluation

The runtime performance of both routing algorithms was evaluated across larger traffic volumes in order to analyse scalability and computational overhead.

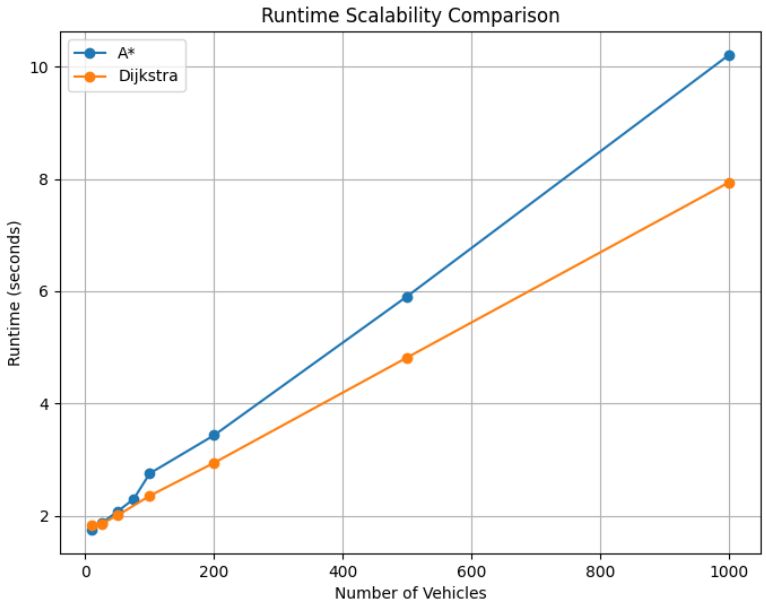

Figure 6.1: Runtime comparison between Dijkstra and A* under heavier network utilisation

Figure 6.1 shows that runtime increased significantly as the number of simulated vehicles became larger.

Vehicles	Dijkstra Runtime (s)	A* Runtime (s)	Runtime Difference (%)
10	1.89	1.88	0.53
50	2.16	2.21	-2.31
100	2.50	2.66	-6.40
200	3.12	3.43	-9.94
300	3.90	4.20	-7.69
400	4.51	5.01	-11.09
1000	8.55	9.40	-9.94

Table 6.1: Runtime comparison between Dijkstra and A* under increasing traffic density

Table 6.1 shows that runtime increased as vehicle density became larger due to an increase in shortest-path computation and congestion-update overhead throughout the graph. While A* theoretically reduces unnecessary graph traversal through heuristic-guided exploration, runtime improvements remained relatively moderate throughout the experiments due to the constrained network and repeated congestion recalculation operations.

Although A* can reduce practical search-space exploration when an informative admissible heuristic is available, the measured runtime difference between Dijkstra and A* remained relatively small throughout several experiments. One major contributing factor was the constrained topology of the network itself. Many vehicle routes converged toward similar arterial roads and high-connectivity intersections regardless of the routing algorithm used.

At lower vehicle volumes, both algorithms maintained relatively stable runtime performance. Once simulations exceeded roughly 500 vehicles, congestion recalculation became noticeably more expensive due to repeated graph traversal operations. Congestion-update overhead gradually became a dominant computational cost relative to shortest-path computation itself under heavier traffic densities.

The experimental results suggest that heuristic-guided routing can improve scalability theoretically, although practical runtime improvements remain strongly constrained by transportation topology and limited route diversity throughout the network. Barthelemy (2011)

6.5 Average Travel Cost Evaluation

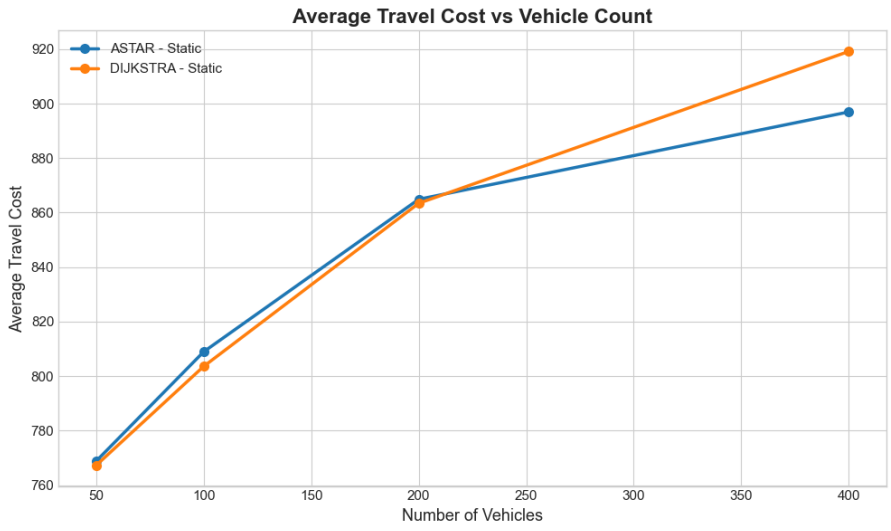

Figure 6.2: Average travel cost comparison between Dijkstra and A* under increasing vehicle density

Figure 6.2 shows the average travel cost produced by both routing algorithms as traffic volume increases throughout the simulation. In both cases, travel cost increased as additional vehicles entered the network, which reflected the growing congestion pressure across high utilisation routes.

Vehicles	Dijkstra Average Cost	A* Average Cost	Relative Difference (%)
50	872.78	698.71	19.95
100	737.40	715.30	3.00
200	770.64	705.84	8.41
300	750.54	751.50	0.96
400	810.05	771.41	4.77

Table 6.2: Average travel-cost comparison between Dijkstra and A*

At lower traffic volumes, both algorithms produced similar route costs. Under heavier traffic conditions, A* consistently generated slightly lower average travel costs than Dijkstra, suggesting that heuristic-guided routing enabled A* to identify alternative low-cost paths more effectively when congestion became more severe.

The difference became more noticeable at larger vehicle counts where congestion weighting increasingly influenced route selection. By incorporating heuristic distance estimation during graph traversal, A* reduced reliance on several heavily congested arterial pathways and distributed traffic more efficiently across lower-utilisation regions of the network.

As congestion multipliers increased throughout the graph, shortest-path optimisation became increasingly dependent on adaptive edge weighting rather than baseline traversal distance alone.

The results suggest that heuristic-guided routing can improve overall traffic efficiency under congestion conditions, although the magnitude of improvement remained constrained by transportation topology and limited route diversity.

This behaviour demonstrates how adaptive edge weighting can alter route-selection behaviour dynamically as congestion evolves throughout the network.

6.6 Average Edge Usage Evaluation

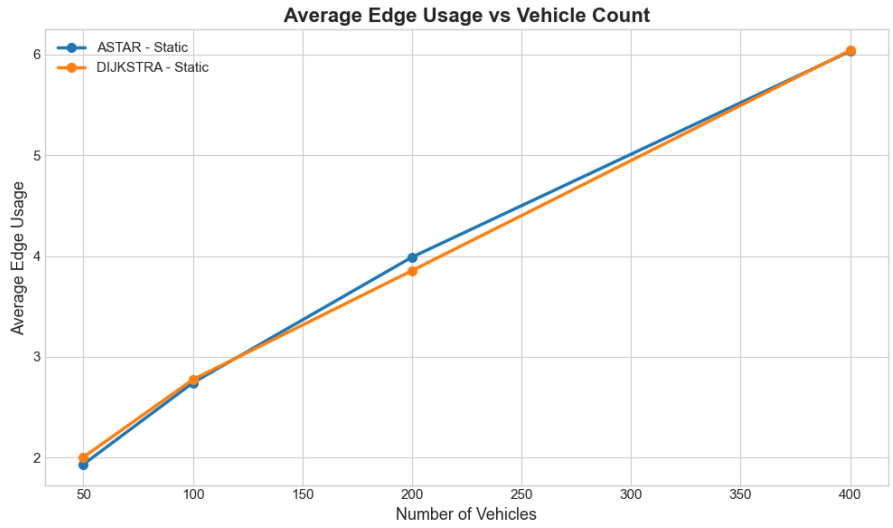

Figure 6.3: Average edge usage comparison between Dijkstra and A* under higher traffic loads

Vehicles	Dijkstra Average Edge Usage	A* Average Edge Usage	Difference (%)
50	313	263	15.97
100	385	353	8.31
200	423	426	-0.71
400	446	453	-1.57

Table 6.3: Average edge-utilisation comparison between Dijkstra and A*

Figure 6.3 shows the average edge utilisation generated by both routing algorithms as traffic volume increased. Average edge utilisation increased as additional vehicles entered the network, reflecting progressively greater congestion pressure across heavily traversed graph regions.

At lower vehicle counts, the difference between the algorithms remained relatively small because sufficient route capacity was available throughout most areas of the network. Under larger traffic loads there was a noticeable difference in traffic distribution behaviour.

A* generally maintained slightly lower average edge utilisation than Dijkstra at higher traffic volumes. Heuristic-guided routing redistributed sections of traffic flow away from high utilisation roads when alternative traversal pathways were available, further reducing the local congestion accumulation throughout several regions of the graph.

As vehicle density increased, adaptive congestion weighting increasingly influenced route selection behaviour throughout the graph. Under these conditions, routing performance became dependent not only on shortest-path distance but also on the ability of the algorithm to avoid congested traversal regions dynamically.

Despite these improvements, both algorithms still produced relatively similar utilisation patterns across major arterial roads and high-connectivity intersections. This indicates that transportation topology remained a dominant factor influencing traffic distribution throughout the network regardless of the routing strategy used.

Heuristic-guided routing produced moderate improvements in traffic balancing under congested conditions. However the overall re-routing effectiveness remained constrained by the structure of the network.

6.7 Adaptive Re-routing Performance

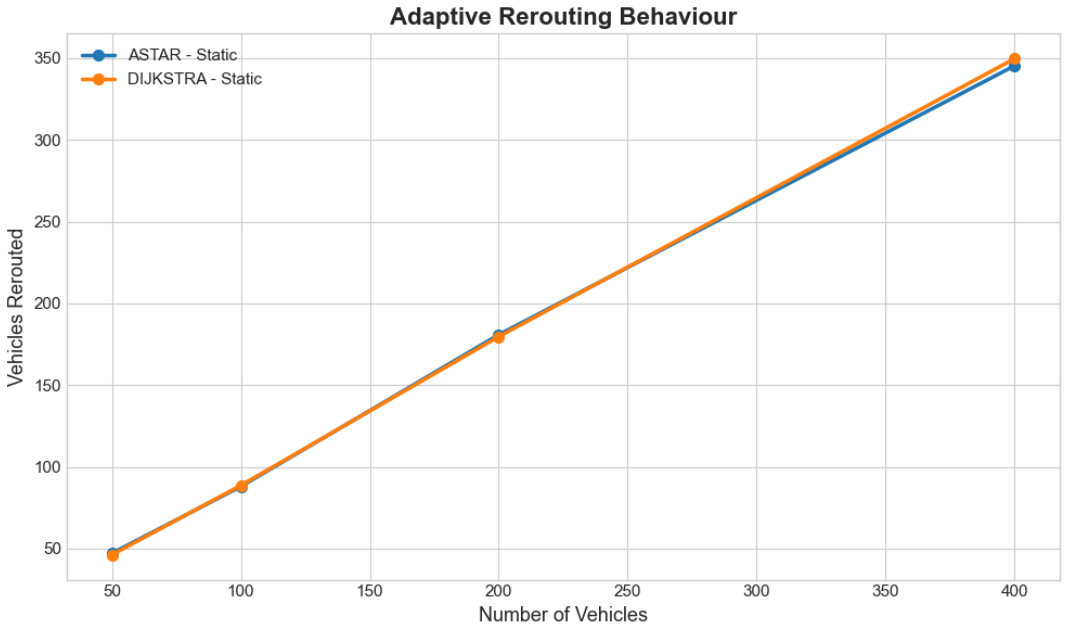

Figure 6.4: Number of adaptively re-routed vehicles under increased traffic volumes

Figure 6.4 shows the number of vehicles which had recalculated alternative routes after congestion weighting had been applied to the graph.

When vehicle density was low few vehicles required route changes because congestion levels remained low across most road segments. As vehicle density increased, the number of re-routed vehicles increased significantly due to greater edge utilisation and higher congestion multipliers being applied throughout the network.

Adaptive congestion weighting altered routing behaviour as vehicle density increased.

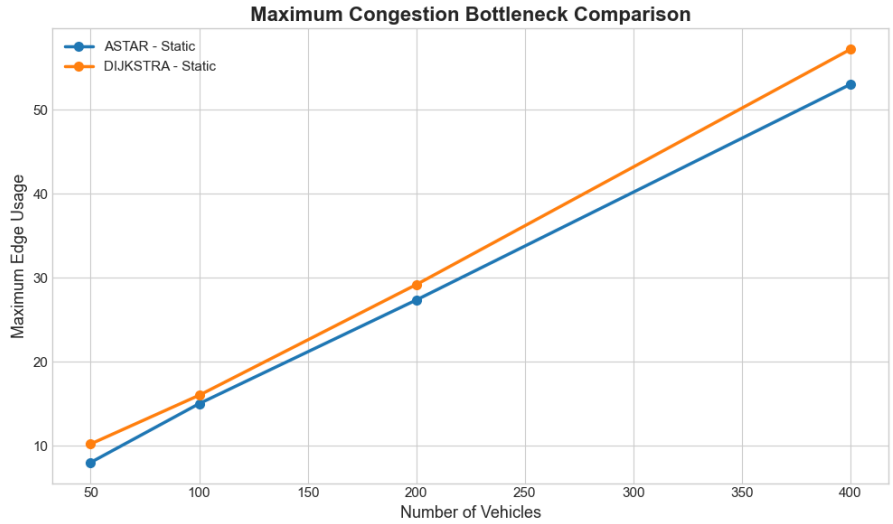

Figure 6.5: Maximum bottleneck severity comparison between Dijkstra and A* under heavier network utilisation

Figure 6.5 shows that maximum edge utilisation increased with vehicle density. This indicates that congestion accumulated disproportionately across a relatively small number of highly utilised roads compared to being distributed evenly throughout the network.

Both Dijkstra and A* produced relatively similar bottleneck behaviour under heavier traffic conditions. Although A* uses heuristic-guided exploration to reduce unnecessary graph traversal, many vehicles still converged toward similar arterial pathways because these routes remained the most efficient traversal regions within the transportation graph.

At lower vehicle volumes, congestion remained relatively distributed across the network. although as a number of simulated vehicles increased, several persistent bottleneck regions emerged where edge utilisation increased rapidly.

Vehicles	Re-routed Vehicles	Max Bottleneck Severity	Runtime (s)
50	10	2.25	2.16
100	21	2.79	2.49
200	32	4.27	3.12
400	57	6.87	4.52

Table 6.4: Adaptive re-routing behaviour under increasing traffic density

The results also demonstrate the importance of transportation topology within routing systems. Even when congestion-aware re-routing was applied, multiple roads continued experiencing high utilisation as the surrounding road structure provided constrained route diversity.

Several bottleneck regions appeared persistently across multiple experiments regardless of the routing algorithm used, suggesting that structural graph constraints influenced congestion formation more strongly than local shortest-path optimisation alone.

The experimental results showed that the number of re-routed vehicles increased as overall vehicle density became larger. At lower traffic volumes, many vehicles retained

identical routes because congestion levels were not severe enough to alter edge costs although larger simulations produced more frequent route recalculations due to increased edge utilisation across the graph as a whole.

6.8 Route Traversal Complexity

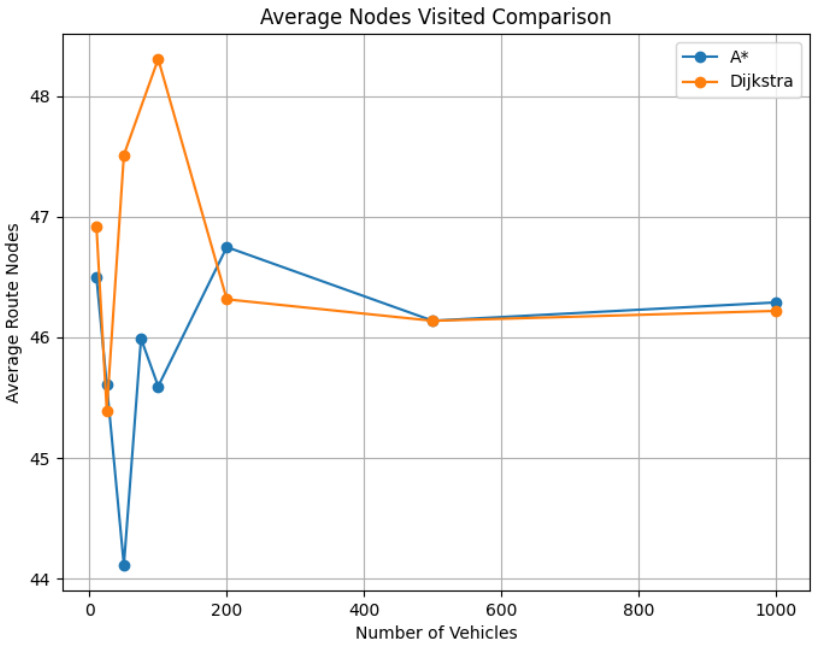

Figure 6.6: Average route traversal complexity comparison between Dijkstra and A*

Figure 6.6 compares the average number of route nodes traversed by Dijkstra and A* across increasing vehicle sizes. Although A* heuristically guides graph traversal towards the destination node, both algorithms converged toward approximately equivalent average route traversal complexity.

At lower vehicle densities, small fluctuations between Dijkstra and A* were observed due to randomised origin-destination generation and local graph topology variations. However, as vehicle density increased, traversal complexity stabilised and both algorithms converged towards approximately equivalent average route sizes.

These fluctuations were expected because localised routing behaviour became more sensitive to individual path-selection differences under lower congestion conditions.

The results suggest that although heuristic-guided search can theoretically reduce unnecessary graph exploration, practical improvements remain strongly dependent on transportation topology, heuristic quality, and constrained route diversity throughout the network.

6.9 Congestion Distribution Analysis

The congestion heat-map visualisation provided insights into how traffic accumulates throughout the network under an increase in vehicle density.

Congestion was not distributed evenly across the graph but instead, traffic was concentrated around arterial roads, major intersections and highways which connected large portions of the network together. The regions consistently produced the highest edge utilisation values throughout the simulation as seen in the heat-maps 2.1 and 2.2

While adaptive re-routing redistributed portions of traffic flow, several bottleneck regions remained persistent across multiple experiments. This occurred because many shortest-path solutions shared similar traversal pathways regardless of the selected algorithm.

In multiple cases, congestion remained severe because vehicles had limited viable alternative routes available within the surrounding road structure.

Several highly utilised intersections additionally appeared to correspond with regions of relatively high graph connectivity and betweenness centrality. These regions naturally attracted larger traffic volumes because many shortest-path routes passed through the same traversal corridors.

6.10 Search Space Exploration Evaluation

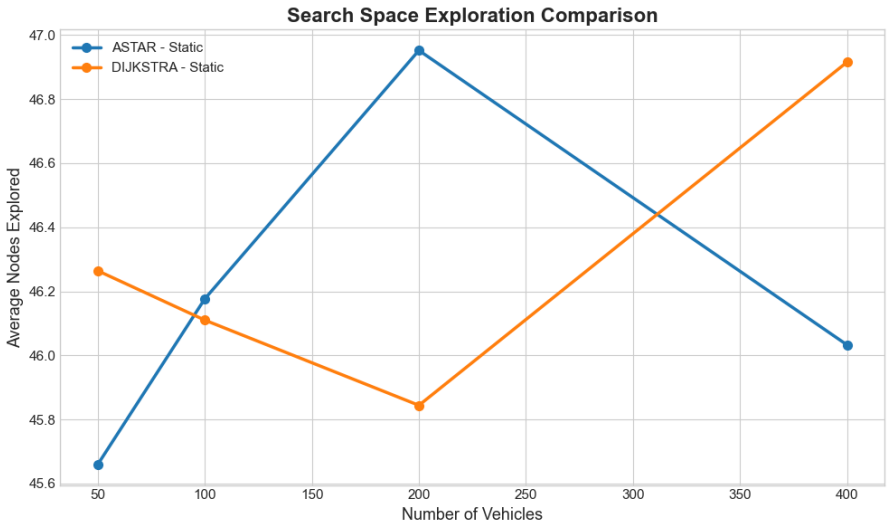

Figure 6.7: Average search-space exploration comparison between Dijkstra and A* under increasing vehicle density

Figure 6.7 compares the average number of explored nodes generated by both routing algorithms during route computation. The number of explored nodes remained relatively stable across increasing traffic volumes, although moderate fluctuations were observed throughout the experiments.

Dijkstra generally explored a slightly larger portion of the graph compared to A*. This behaviour was expected because Dijkstra performs uninformed graph traversal whereas A* uses heuristic distance estimation to guide exploration toward the destination node more efficiently.

In practice, the performance gap between Dijkstra and A* remained relatively small across most experiments. One contributing factor was the constrained structure of the network itself. Many shortest-path solutions converged toward similar high-connectivity traversal corridors regardless of the routing strategy used, reducing the practical effectiveness of heuristic-guided exploration.

Despite this limitation, the results suggest that heuristic-guided traversal reduced unnecessary graph exploration under evolving congestion. The reduction in search-space complexity contributed to improved route-selection efficiency despite the additional computational overhead associated with heuristic evaluation.

The results additionally demonstrate that heuristic effectiveness within transportation routing problems is strongly dependent on graph topology, route diversity, and the structural connectivity of the surrounding road network.

6.11 Neural Network Evaluation

The neural network was evaluated using multiple regression metrics in order to assess prediction accuracy, convergence behaviour, and generalisation capability under congestion conditions.

MSE was selected because larger prediction errors are penalised quadratically.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 \tag{6.1}$$

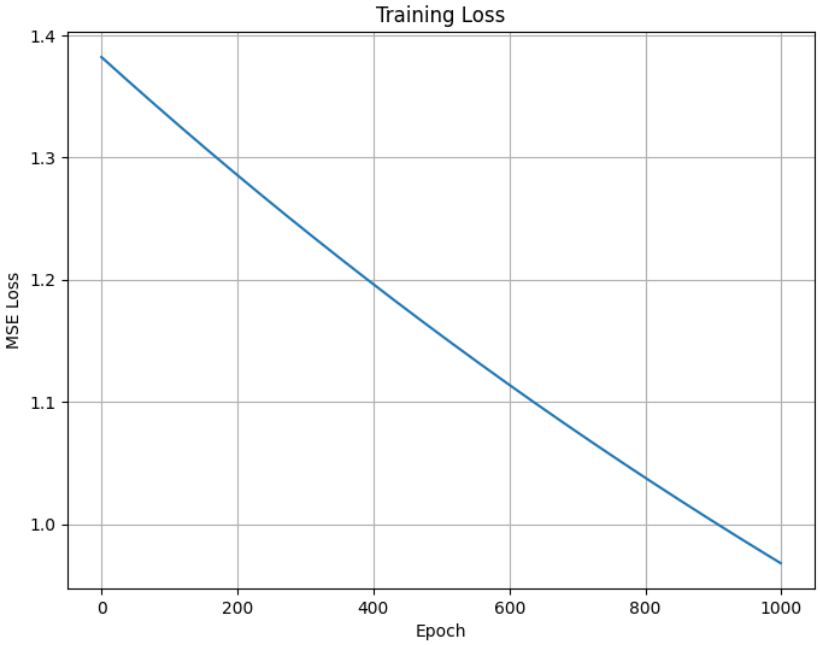

Figure 6.8: Neural network training loss across epochs

Figure 6.8 reveals that the neural network achieved stable convergence throughout training. The Mean Squared Error (MSE) decreased consistently across epochs, indicating that back-propagation and gradient descent successfully optimised the network weights. The gradual reduction in loss also suggests that the model was capable of learning congestion relationships from the training dataset without unstable divergence behaviour.

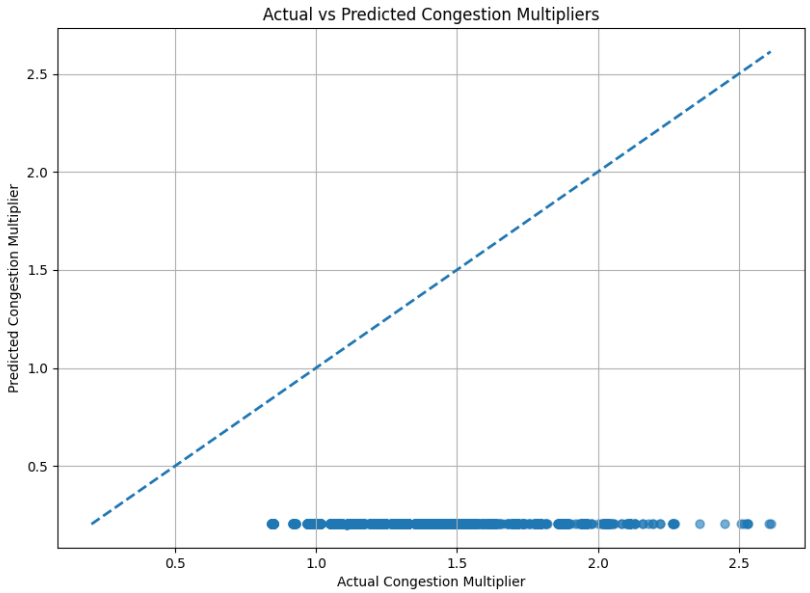

Figure 6.9: Actual versus predicted congestion multipliers generated by the feed-forward neural network

Figure 6.9 compares the predicted congestion multipliers against the true congestion values from the training dataset. While the neural network successfully learned general congestion behaviour, the model demonstrated a tendency to predict values clustered around lower congestion states. This indicates that the network under-fit higher congestion conditions and struggled to model more severe non-linear traffic patterns.

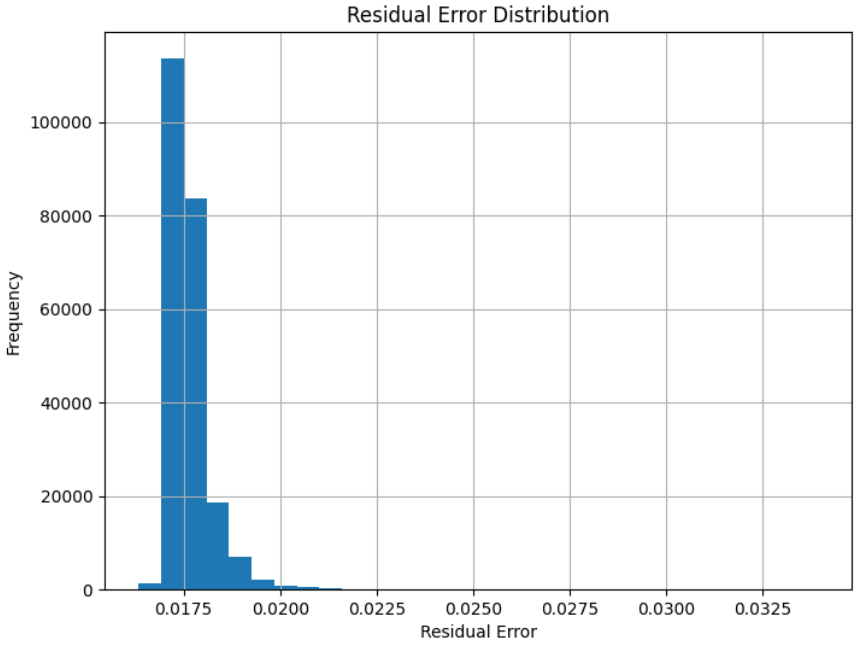

Figure 6.10: Residual error distribution generated by the feed-forward neural network

Actual Congestion	Predicted Congestion	Absolute Error
0.32	0.29	0.03
0.51	0.47	0.04
0.75	0.58	0.17
0.91	0.63	0.28

Table 6.5: Example congestion predictions generated by the neural network

Figure 6.10 shows the distribution of residual prediction errors produced by the neural network during congestion prediction. Residual error represents the difference between the true congestion multiplier and the predicted congestion multiplier generated by the model.

Most residual values were concentrated within a relatively narrow range which indicates that the network maintained generally consistent prediction behaviour across the dataset. Although the residual distribution was not perfectly centred around zero and demonstrated a noticeable positive skew.

The neural network systematically under-predicted congestion severity for a portion of the dataset, especially under higher congestion conditions. The model therefore struggled

to fully capture more complex traffic relationships despite successfully learning broader congestion trends.

The residual distribution additionally indicates that prediction behaviour remained relatively stable without significant outlier divergence.

A contributing factor was the imbalance present within the training dataset. Lower congestion multipliers dominated the dataset compared to higher congestion multipliers, causing the network to bias predictions toward lower-congestion classes in order to minimise the total Mean Squared Error. As a result, prediction accuracy for highly congested traffic states was reduced.

Metric	Value
Mean Squared Error (MSE)	0.089480
Root Mean Squared Error (RMSE)	0.299132
Mean Absolute Error (MAE)	0.113049
Coefficient of Determination (R^2)	-0.091710

Table 6.6: Neural-network regression evaluation metrics

The Mean Absolute Error of approximately 0.11 indicates that the predicted congestion multiplier typically deviated only slightly from the target value on average. The feed-forward neural network successfully captured broader congestion trends despite the complexity and variability present within dynamically evolving traffic conditions.

The regression metrics indicate that the neural network successfully learned broader congestion relationships from the generated traffic dataset despite the complexity of dynamically evolving transportation behaviour. Prediction accuracy remained strongest under lower and moderate congestion conditions, although performance decreased under highly congested states where traffic behaviour became increasingly non-linear and structurally constrained.

Although the R^2 score remained slightly negative, the neural network still demonstrated stable convergence behaviour and relatively low average prediction error throughout experimentation. This limitation emerged partly because the implemented feed-forward architecture lacked temporal memory and therefore could not model sequential traffic dependencies or evolving congestion propagation across time.

The model nevertheless learned meaningful congestion relationships and produced sufficiently stable congestion estimates capable of influencing adaptive route-selection behaviour within the transportation simulation.

Prediction accuracy decreased under heavily congested traffic states where congestion multipliers became increasingly non-linear. This behaviour is visible in Figure 6.9, where predicted values progressively deviated from the ground-truth distribution at higher congestion levels.

Mean Squared Error was selected as the primary optimisation objective because larger prediction errors are penalised more heavily than smaller deviations. This was particularly important within congestion prediction because inaccurate estimation of highly congested

traversal regions can significantly influence adaptive route-selection behaviour and overall traffic distribution throughout the transportation network.

Chapter 7

DISCUSSION

7.1 Routing Performance Analysis

The experimental evaluation demonstrated that both Dijkstra's algorithm and A* were capable of efficiently generating shortest-path routes across the Oxford network. As expected, A* generally produced slightly improved runtime performance and lower average edge utilisation compared to Dijkstra due to the introduction of heuristics.

A* theoretically reduces unnecessary graph traversal through admissible heuristic estimation but as the results indicated, performance improvements remained relatively moderate throughout the experiments. One of the primary contributing factors was the constrained topology of the network itself. Many shortest-path solutions converged toward similar high-connectivity traversal corridors regardless of the routing strategy used. As a result, the Euclidean heuristic used within A* provided only moderate practical improvements because route diversity throughout several regions of the network remained structurally limited.

These findings suggest that transportation topology may influence large-scale traffic behaviour more strongly than routing strategy alone within structurally constrained urban environments Barthelemy (2011). Within regions containing limited alternative traversal pathways, congestion accumulated regardless of the routing algorithm used showing that the effectiveness of routing systems is constrained not only by algorithmic design but also by the underlying graph structure of the transportation network

The project demonstrated that congestion prediction quality is strongly influenced by simulation realism and congestion dynamics.

Initial implementations used handcrafted threshold-based congestion multipliers, producing artificially simple prediction targets. Although these targets were easier for the neural network to approximate, they did not accurately reflect emergent traffic behaviour.

This highlighted an important trade-off between predictive simplicity and simulation realism throughout congestion modelling.

Later iterations replaced discrete congestion thresholds with smooth congestion scaling and simulation-derived congestion ratios. This produced more realistic adaptive traffic

behaviour, but also significantly increased prediction complexity and reduced neural-network performance metrics.

These findings suggest that realistic traffic prediction systems depend heavily on temporal congestion modelling, evolving vehicle interactions, and richer representations of network-wide traffic propagation rather than neural-network complexity alone.

Metric	Dijkstra	A*	Observation
Runtime Complexity	Higher	Lower	Moderate heuristic-routing improvement
Average Travel Cost	Higher	Lower	Improved congestion-aware routing
Search-Space Exploration	Larger	Smaller	Reduced graph traversal complexity
Edge Utilisation	Higher	Lower	Improved traffic distribution
Bottleneck Severity	Similar	Similar	Topology remained dominant
Adaptive Re-routing	Effective	Effective	Congestion weighting altered routing behaviour

Table 7.1: Overall routing-performance comparison between Dijkstra and A*

7.2 Congestion Behaviour and Bottleneck Formation

The persistence of bottleneck regions despite re-routing decisions details that congestion emergence was driven more strongly by transportation topology than by shortest-path inefficiency alone.

Several bottleneck regions consistently emerged across multiple experiments involving different vehicle densities. These bottlenecks were especially visible on motorway-like structures and major connecting roads where alternative viable routes were limited. Even after adaptive congestion weighting and dynamic re-routing were introduced, many vehicles continued converging toward similar high-capacity roads due to the lack of feasible alternative traversal pathways.

This behaviour highlights an important limitation of shortest-path-based traffic optimisation systems. Edge weighting successfully redistributed portions of traffic flow throughout the network, although the surrounding transportation topology constrained the overall effectiveness of adaptive optimisation. Many transportation systems contain structurally centralised roads which naturally attract large traffic volumes regardless of re-routing.

Adaptive routing reduced localised congestion and improved traffic distribution throughout the network, however consistent bottleneck regions remained under heavily constrained road networks which indicates that optimisation cannot fully eliminate congestion where routes are less diverse.

From a graph-theoretic perspective, several bottlenecks appeared to correspond closely with regions with relatively high betweenness centrality and graph connectivity. These intersections naturally attracted larger traffic volumes because many shortest-path solutions converged through the same structurally central traversal corridors. This suggests that congestion is influenced not only by routing strategy but also by the underlying topological structure of the graph itself.

The findings therefore suggest that adaptive routing effectiveness is fundamentally constrained by graph topology, route diversity, and the structural organisation of the transportation network itself.

7.3 Adaptive Re-routing Effectiveness

The implementation of route adaptation successfully altered vehicle-routing behaviour throughout the network. As edge utilisation increased, congestion multipliers increased the effective travel-time costs of overloaded roads which encouraged vehicles to explore alternative traversal pathways across the graph.

Experimental evaluation showed that the number of re-routed vehicles increased significantly as traffic density became larger. Under lower traffic conditions, most vehicles retained identical shortest-path routes because congestion levels remained insufficient to justify alternative traversal pathways. Larger simulations involving hundreds of vehicles produced substantially more route recalculations due to the increased congestion weighting applied throughout the network.

The results demonstrate that adaptive congestion weighting dynamically altered shortest-path optimisation behaviour as surrounding traffic conditions evolved. Rather than minimising static traversal distance alone, route selection increasingly depended on congestion-adjusted edge costs and surrounding network utilisation.

Despite these improvements, adaptive re-routing effectiveness remained constrained by transportation topology and route diversity. Several highly utilised arterial corridors continued attracting large traffic volumes even after dynamic weighting was introduced because alternative traversal pathways remained structurally limited throughout several regions of the graph.

These findings suggest that adaptive re-routing can improve traffic redistribution and reduce local congestion severity, although the effectiveness of dynamic optimisation remains fundamentally constrained by the underlying graph structure of the network.

7.4 Neural Network Performance and Prediction Behaviour

The feed-forward neural network successfully learned non-linear relationships between traffic variables and congestion multipliers throughout the generated transportation dataset. The gradual reduction in Mean Squared Error during training showed that the

back-propagation and gradient descent implementation operated correctly and achieved stable convergence behaviour.

The network was capable of modelling lower and moderate congestion conditions effectively but prediction accuracy decreased under more heavily congested traffic states. Residual-error analysis revealed that the model systematically under-predicted several high-congestion samples which suggests that the network struggled to fully model more complex congestion relationships.

One contributing factor was the imbalance present within the generated training dataset. Lower congestion states appeared substantially more frequently than severe congestion states which biased the network toward predicting lower congestion multipliers in order to minimise the total loss function. This meant that the model was effective across common traffic conditions while producing reduced accuracy for extreme congestion events.

The limitations observed throughout evaluation also reflected several architectural constraints associated with standard feed-forward neural networks. The implemented model operated as a static regression system without temporal memory which limited its ability to model sequential traffic dependencies and evolving congestion propagation across time. Real-world transportation systems are highly dynamic where previous congestion states strongly influence future traffic behaviour through cascading network interactions.

Although more advanced architectures such as recurrent neural networks, graph neural networks or reinforcement learning systems may improve predictive performance and routing behaviour, the feed-forward implementation remained computationally feasible and aligned closely with the objectives of the dissertation. The primary objective of the project was not to maximise predictive accuracy alone, but instead to investigate how learned congestion prediction could influence adaptive route-selection behaviour within a dynamically evolving transportation simulation.

7.5 Scalability and Computational Complexity

Experimental evaluation demonstrated that runtime increased progressively as vehicle density became larger due to the growing number of shortest-path calculations, congestion updates, and adaptive re-routing operations required throughout the transportation graph. Although heuristic-guided routing improved search-space efficiency moderately, large-scale urban traffic simulation remained computationally expensive under heavier traffic densities.

The constrained topology of the network additionally influenced routing scalability. Many shortest-path solutions converged toward similar high-connectivity traversal corridors which increased congestion-update overhead across heavily utilised graph regions. These findings suggest that transportation topology influences not only traffic distribution but also the computational complexity of routing systems themselves.

Sparse graph structures can significantly affect shortest-path complexity, congestion formation, and routing scalability Vlahogianni et al. (2014a). Future implementations

involving real-time routing, larger geographical regions, or live congestion prediction would likely require additional optimisation techniques such as distributed processing, GPU acceleration, parallel graph computation, or hierarchical routing strategies in order to remain computationally feasible at scale.

The final implementation additionally demonstrated the importance of dataset construction and target engineering within machine-learning systems. Earlier implementations using unstable congestion targets produced inconsistent regression behaviour and severely negative evaluation metrics. By introducing smoother congestion scaling, feature standardisation, and target normalisation, the neural network achieved substantially more stable convergence behaviour and improved predictive consistency.

These findings suggest that predictive performance within congestion-modelling systems depends not only on neural-network architecture but also on simulation realism, target stability, and the structural complexity of the surrounding transportation network.

7.6 System Limitations

7.6.1 Static Congestion Modelling

The congestion model relied on simulated edge utilisation rather than real-time transportation data. While this enabled controlled experimental evaluation, real-world traffic systems continuously evolve according to accidents, traffic lights, weather conditions and human driving behaviour which means that the implemented congestion model represents an approximation of real traffic-flow behaviour rather than a fully real-time transportation system.

7.6.2 Synthetic Vehicle Generation

Vehicle origin-destination pairs were generated randomly across the Oxford road network which enabled scalable congestion simulation and generated traffic patterns may not fully represent realistic distributions observed within real transportation systems where traffic demand is heavily concentrated around employment centres, schools and transport roads.

7.6.3 Road-Network Constraints

Several highly utilised roads continued accumulating congestion despite changes in the routes because of the surrounding roads provided limited viable pathways, Highlighting the transportation structure itself constrains the effectiveness of local routing optimisation techniques.

Chapter 8

CONCLUSION

This dissertation investigated congestion-aware adaptive traffic routing using graph optimisation and neural-network-based congestion prediction within a real-world transportation environment.

The project successfully implemented a complete traffic-routing and congestion-modelling framework using road-network data extracted from OpenStreetMap through the OSMnx library. The network was represented as a weighted directed graph where intersections corresponded to graph nodes and roads corresponded to weighted edges containing travel-time and congestion information.

Classical shortest-path routing algorithms including Dijkstra's algorithm and A* were then evaluated. Experimental results demonstrated that both algorithms were capable of efficiently generating routes across the transportation graph, although A* generally produced moderate improvements in runtime efficiency, average edge utilisation and travel-cost optimisation due to heuristic-guided search behaviour.

Congestion weighting transformed the routing system from a static shortest-path problem into an adaptive weighted optimisation problem. Edge weighting successfully altered routing behaviour by increasing traversal costs on heavily utilised roads and encouraging vehicles to explore viable alternative pathways.

Route adaptation improved traffic distribution across parts of the network. However, congestion could not be fully eliminated because several roads acted as unavoidable bottlenecks. Several roads and highly connected intersections consistently formed bottleneck regions because many shortest-path solutions converged toward similar traversal pathways regardless of re-routing.

A custom feed-forward neural network was additionally implemented entirely from first principles without external machine-learning frameworks. The network successfully learned relationships between traffic features and congestion multipliers using forward propagation, back-propagation and gradient descent optimisation.

Regression evaluation demonstrated stable training convergence and reasonable predictive performance across lower and moderate congestion conditions but prediction accuracy decreased under more severe congestion states due to the increased non-linearity of traffic behaviour and imbalance within the generated dataset. Despite these limitations, the

neural-network system demonstrated how learned congestion prediction can influence adaptive traffic-routing behaviour within transportation systems.

Overall, the dissertation demonstrated that congestion-aware adaptive routing combined with graph-based optimisation and neural-network prediction can improve traffic redistribution within dynamically weighted transportation systems. Although structural road-network constraints limited the extent to which congestion could be eliminated entirely, the experimental evaluation showed that adaptive edge weighting and heuristic-guided routing were capable of reducing local congestion severity and improving routing efficiency under higher traffic densities. The findings additionally highlighted the importance of transportation topology, congestion modelling realism, and scalable optimisation techniques within intelligent transportation research.

Chapter 9

FUTURE WORK

Several improvements and extensions could further enhance the scalability, realism and predictive capability of the proposed adaptive traffic-routing system.

One major improvement would involve integrating real-time traffic data through external transportation APIs. The current implementation relied on simulated congestion which is generated from edge utilisation statistics. Incorporating live traffic conditions would allow the system to operate within evolving transportation environments and improve real-world applicability.

Future research could investigate more advanced neural-network architectures for congestion prediction. The implemented feed-forward neural network operated as a static regression model and lacked temporal memory. Architectures such as recurrent neural networks (RNNs), long short-term memory networks (LSTMs) and graph neural networks (GNNs) may capture temporal traffic dependencies and spatial relationships between connected road segments better.

Graph neural networks may be particularly suitable for transportation systems because road networks already exist in graph form. Message-passing operations across road segments may improve the modelling of network-wide congestion propagation and dynamic traffic-flow behaviour.

Reinforcement learning could also provide a stronger optimisation framework for adaptive traffic routing Sutton and Barto (2018). Rather than predicting congestion statically, reinforcement-learning agents could continuously learn routing policies through interaction with the transportation environment which could improve long-term traffic balancing and adaptive decision-making under changing traffic conditions.

The simulation environment itself could be extended through more realistic vehicle behaviour modelling. Future implementations may include:

- traffic-light systems,
- lane-level simulation,
- weather conditions,
- accidents and road closures,

- variable driver behaviour,
- real-world traffic demand distributions

These additions would improve simulation realism and allow investigation of more complex transportation scenarios.

Scalability also remains an important area for future optimisation. Larger transportation networks and real-time traffic systems would require significantly greater computational resources. If the system were expanded to larger cities, additional optimisation techniques such as GPU acceleration or parallel graph processing would probably become necessary.

Another potential extension would involve integrating multi-objective optimisation techniques. The current system primarily optimised congestion-adjusted travel cost although future systems may simultaneously optimise:

- fuel efficiency,
- carbon emissions,
- travel time,
- road safety,
- traffic balancing,
- environmental impact

Finally, future work could investigate deployment within larger geographical regions using real-world transportation datasets. Expanding the system beyond the Oxford road network would allow evaluation under more diverse transportation structures and traffic conditions while improving the generalisability of the experimental findings.

Overall, the project provides a strong foundation for further research into intelligent transportation systems, adaptive congestion optimisation and AI-driven traffic-management solutions.

Appendices

.1 Routing Algorithm Pseudocode

The following pseudocode summarises the routing workflow implemented throughout the transportation system.

Algorithm 1 Congestion-Aware Adaptive Routing

```
Generate transportation graph G(V, E)
Generate valid origin-destination vehicle pairs
for each vehicle do
    Compute shortest-path route using Dijkstra or A*
end for
Calculate edge utilisation across the graph
for each edge do
    Apply congestion multiplier to travel-time weight
end for
for each vehicle do
    Recalculate shortest-path route using updated weights
end for
Generate congestion statistics and evaluation metrics
Train neural network using generated congestion dataset
Predict congestion multipliers using trained model
```

The routing workflow combines graph-based optimisation, edge weighting, and neural-network-based congestion prediction within a transportation simulation framework.

Bibliography

G. Arulampalam and A. Bouzerdoum. A generalized feedforward neural network architecture for classification and regression. *Neural Networks*, 16(5–6):561–568, 2003. doi: 10.1016/S0893-6080(03)00116-3.

M. Barthelemy. Spatial networks. *Physics Reports*, 499(1–3):1–101, 2011.

G. Boeing. Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017. doi: 10.1016/j.compenvurbsys.2017.05.004.

A. Calatayud, S. Sánchez González, F. Bedoya-Maya, F. Giraldez, and J. M. Márquez. Urban road congestion in latin america and the caribbean: Characteristics, costs, and mitigation. Technical Report IDB-TN-2145, Inter-American Development Bank, 2021. URL `https://publications.iadb.org/publications/english/document/Urban-Road-Congestion-in-Latin-America-and-the-Caribbean-Characteristics-Costs-and-pdf`.

A. Candra, M. A. Budiman, and K. Hartanto. Dijkstra's and a-star in finding the shortest path: a tutorial. pages 28–32, 2020. doi: 10.1109/DATABIA50434.2020.9190342.

Y. Chang, Y. Lee, and S. Choi. Is there more traffic congestion in larger cities? scaling analysis of the 101 largest u.s. urban centers. *Transport Policy*, 59:54–63, 2017. doi: 10.1016/j.tranpol.2017.07.002.

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 2009.

E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

D. Foead, A. Ghifari, M. Kusuma, N. Hanafiah, and E. Gunawan. A systematic literature review of a* pathfinding. *Procedia Computer Science*, 179:507–514, 2021. doi: 10.1016/j.procs.2021.01.034.

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

B. D. Greenshields. A study of traffic capacity. *Highway Research Board Proceedings*, 14: 448–477, 1935.

A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.

P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2): 100–107, 1968a.

P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2): 100–107, 1968b.

J. J. Hopfield. Neural networks and physical systems with emerent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, 1982. doi: 10.1073/pnas.79.8.2554.

N. Kokash. An introduction to heuristic algorithms. *Department of Informatics and Telecommunications*, 1(2005):1–7, 2005.

A. Krogh. What are artificial neural networks? *Nature Biotechnology*, 26(2):195–197, 2008. doi: 10.1038/nbt1386.

T. S. Standley. Finding optimal solutions to cooperative pathfinding problems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 24, pages 173–178, 2010. doi: 10.1609/aaai.v24i1.7564.

R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

E. I. Vlahogianni, M. G. Karlaftis, and J. C. Golias. Short-term traffic forecasting: Overview of objectives and methods. *Transport Reviews*, 24(5):533–557, 2004. doi: 10.1080/0144164042000195072.

E. I. Vlahogianni, M. G. Karlaftis, and J. C. Golias. Short-term traffic forecasting: Where we are and where we're going. *Transportation Research Part C*, 43:3–19, 2014a.

E. I. Vlahogianni, M. G. Karlaftis, and J. C. Golias. Short-term traffic forecasting: Where we are and where we're going. *Transportation Research Part C*, 43:3–19, 2014b.

J. G. Wardrop. Some theoretical aspects of road traffic research. In *Proceedings of the Institution of Civil Engineers*, volume 1, pages 325–362, 1952.

Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021a. doi: 10.1109/TNNLS.2020.2978386.

Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu. A comprehensive survey on graph neural networks for traffic prediction. *IEEE Transactions on Intelligent Transportation Systems*, 22(1):1–21, 2021b.